

Seasons in Teamder — *continued*

3. How a מחזור is counted — the three counters

A מחזור (*machzor*) is one game-night. This club plays one a week. A משהקון (*mischakon*) is one mini-game inside that night; there are typically four to six of them, and they exist only in "advanced mode". The one club that has ever run a season is on the plain timer: across its ten sealed evenings it recorded `stats.rounds` of 4, 2, 0, 3, 2, 4, 0, 0, 0, 0 — and zero on every evening since the feature was switched on. Everything in this section counts **evenings**. Where a number in the codebase is called `rounds` and means mini-games, it is named as such.

The app holds three independent, separately-written answers to "how many evenings has this club played", plus four server derivations and two frozen archive copies. None of them is a view of another. This section is the map of all of them.

The three writers

#	Where it lives	Written by	Unit	Resets?	Client can read it?
1	<code>clubRecords/{groupId}.eveningsSealed</code>	server, <code>FieldValue.increment(1)</code>	evenings sealed since 26.08.2026	never	no — rules deny
2	<code>groups/{groupId}.seasons.playedRounds</code>	server, seeded + <code>increment(1)</code>	evenings in the running season	zeroed by every season-opening path	yes
3	nothing — derived on the phone	<code>gameService.getCommunityStats</code> scan	evenings in the 200-newest terminal games	n/a (recomputed each read)	it is the client

Counter 1 — `clubRecords.eveningsSealed`

The oldest of the three, and the only one that is *exact*. It is written in one place, `functions/src/index.ts:5194`, inside the batch at the bottom of `sealRoundSummary`:

```
// functions/src/index.ts:5188–5196
batch.set(
  db.collection('clubRecords').doc(groupId),
  {
    groupId,
    ...next,
    // INCREMENT, not `read + 1`.
    eveningsSealed: admin.firestore.FieldValue.increment(1),
```

The comment above it records the incident that forced the atomic form: three evenings sealed within a minute on the QA club left three summaries and a counter of two. It is safe to increment because `summaryRef.create()` at `:5162` throws if this evening was already sealed, so the batch runs at most once per game.

Three properties matter, and all three are traps.

It began on **26.08.2026**. `sealRoundSummary` shipped in commit `59cd9b6` (2026-08-25). Production proves the date exactly: `clubRecords/HhzIwmjMl1i5HS0Ght3p` carries `since: 1787739600000 = 2026-08-26T10:20:00Z`, which is the `startsAt` of `games/T7qRTLr0...`, the club's first sealed evening. Every evening before it — twelve of them, from 28.06 to 24.08 — has **no** `roundSummaries` **document at all** and is invisible to this counter. I enumerated the club's 23 terminal games and probed `roundSummaries/{gameId}` for each:

Evening (UTC)	roundSummary	basis.eveningsCompared	mini-games
28.06 – 24.08 (12 evenings)	absent	—	—
26.08 10:20	present	0	4
31.08 13:35	present	1	2
07.09 05:25	present	2	0
08.09 06:35	present	3	3
11.09 05:35	present	4	2
14.09 09:40	present	5	4
15.09 13:00	present	6	0
16.09 06:20	present	7	0
17.09 14:30	present	8	0
17.09 14:35	present	9	0
17.09 21:19	absent (auto-closed, playVerified:false)	—	—

`eveningsCompared` is `eveningsSealed` as read at the top of that seal (`index.ts:5153`), so the ladder 0→9 is the counter's own history. It stands at **10** today. The club has played **22**.

Nothing decrements it. The deletion handler at `index.ts:5357–5391` writes a `gameDeletions` audit row and sends pushes; it does not touch `clubRecords` or `seasons`. Delete a sealed evening and the counter keeps it. This is deliberate — `src/utils/seasonLifecycle.ts:34–37` states it as the reason to prefer the counter over a query — but it means counter 1 and counter 3 drift permanently apart on any deletion, in the opposite direction from the 26.08 blindness.

The client cannot see it. `firestore.rules:1813` is `allow read, write: if false`. This is why counter 2 exists at all.

Counter 2 — seasons.playedRounds

The mirror. It is the number the club card shows, the number the hourly sweep closes on, and the number `endSeasonNow` archives. `src/components/community/SeasonsCard.tsx:151`:

```
<Text style={styles.big}>{seasons.playedRounds ?? 0}</Text>
<Text style={styles.bigOf}>{he.seasonsCardOfTarget(cadence!.targetRounds as number)}</Text>
```

`he.seasonsCardOfTarget` (`src/i18n/he.ts:2126`) renders מתוך {target} מחזורים ("out of {target} evenings"), so the card reads e.g. "מתוך 24 מחזורים 0".

It is **seeded** at activation and **incremented** at every qualifying seal (`index.ts:5216–5236`):

```
if (seasonRoundsAtStart !== null) {
  batch.set(
    db.collection('groups').doc(groupId),
    { seasons: { playedRounds: admin.firestore.FieldValue.increment(1) } },
    { merge: true },
  );
}
```

`seasonRoundsAtStart` is non-null only when the evening belongs to the *running* season (`index.ts:5092–5106`):

```
if (sea?.enabled) {
  tableZeroedAt = num(sea.roundsAtStart);
  const stamp = typeof args.seasonId === 'string' ? args.seasonId : '';
  const mine = stamp ? stamp === sea.currentId : sea.currentNo === 1;
  if (mine) seasonRoundsAtStart = tableZeroedAt;
  else { console.log('[season] seal is not for the running season – progress not credited', ...); }
}
```

That `mine` expression is the stamp rule (below), and it is why an admin confirming a forgotten evening from an archived season no longer pushes the current season one evening closer to its target.

The **seed** comes from `src/utils/seasonSeed.ts` (mirrored at `functions/src/seasonSeed.ts`), called at `index.ts:16572–16575`:

```
...seasonSeed(
  await sealedEveningsOf(groupId), // → roundsAtStart
  closedSoFar > 0 ? 0 : await playedEveningsFromGames(groupId), // → playedRounds
),
```

so season 1 of a first-time club is seeded with the **games count**, and every later season with **0**. Season 1 owns the club's history; season 5 does not.

Every path that opens a season zeroes it: `index.ts:15905` (the rollover's lifecycle write), `:16501` (the `sealNow` activation branch), `:17057` (`endSeasonNow`). The `continue` activation branch seeds it via `seasonSeed`. The reopen recomputes it from the games (`:16925–16934`).

Because the field is zeroed by three paths and written by `merge:true set()`s, `disable → re-enable` on a club with a running season re-seeds `playedRounds: 0` even though the disable dialog promises `הספירה תמשיך` ("the count will continue from where it stopped") — LEDGER `disable-reenable-zeroes-running-season`. This is not hypothetical on the live club: `seasons.startedAt` is `2026-09-17T21:05:41Z`, five hours *after* season 1 closed at 16:12Z, which is the signature of exactly that re-open.

What happens when it is absent. `completedRoundsFrom` (below) falls through to counter 1 minus the offset. That fallback is the single most expensive line in the feature's history.

Counter 3 — the client scan

`src/services/gameService.ts:949`, `getCommunityStats(groupId, season?)`. One query, one pass, two tallies:

```
// src/services/gameService.ts:1112–1118
const q = query(col.games(), where('groupId','==',groupId),
  where('status','in',['finished','cancelled']),
  orderBy('startsAt','desc'), limit(200));
```

```
// src/services/gameService.ts:1150–1186 (trimmed)
for (const doc of snap.docs) {
  const g = doc.data();
  {
    // — LIFETIME, before the season gate
    const st = eveningPlayState(g as PlayableEvening);
    if (g.status === 'cancelled' || st === 'notHappened') lifeCancelled += 1;
    else if (st !== 'unverified') { lifeFinished += 1; /* ...attendance... */ }
  }
  if (!inSeason(g as { seasonId?: string }, season)) continue;
  const state = eveningPlayState(g as PlayableEvening);
  if (g.status === 'cancelled') { totalCancelled += 1; continue; }
  if (state === 'unverified') continue; // counts as NEITHER half of the rate
  if (state === 'notHappened') { totalCancelled += 1; continue; }
  totalFinished += 1;
  ...
}
```

Two exported rules do all the deciding — `inSeason` (`src/utils/seasonScope.ts:28`) and `eveningPlayState`. Neither is re-implemented here, which is new: until 18.09 the season rule was a closure inside this function, and the only test of it was a copy of the rule re-declared inside `tests/logic/seasonScopedStats.test.ts` (LEDGER `inseason-tested-as-a-private-copy`).

`totalFinished` is the season-scoped answer; `lifetime.totalFinished` is the all-time one. The screen picks between them at `src/screens/communities/CommunityStatsScreen.tsx:769`:

```
value={scopedCard ? scopedCard.completedRounds
  : scope.k === 'all' ? (stats?.lifetime?.totalFinished ?? 0)
  : (stats?.totalFinished ?? 0)}
label={he.communityStatsEvenings} // 'מחזורים'
```

so one tile can render three different numbers for the same club depending on which chip is selected — by design, but see the matrix.

`roundsAtStart` — the offset, and why its source must match the counter's

`roundsAtStart` exists only to make counter 1 usable. Counter 1 never resets, so a season must remember its own zero; `completedRoundsFrom` subtracts:

```
// functions/src/seasonCounters.ts:105-118
export function completedRoundsFrom(
  sealedEvenings: number, roundsAtStart?: number, playedRounds?: number,
): number {
  if (typeof playedRounds === 'number' && playedRounds >= 0) return playedRounds;
  const all = typeof sealedEvenings === 'number' && sealedEvenings > 0 ? sealedEvenings : 0;
  const base = typeof roundsAtStart === 'number' && roundsAtStart > 0 ? roundsAtStart : 0;
  return Math.max(0, all - base);
}
```

The invariant the subtraction needs is: `roundsAtStart` **must be a reading of the same counter, taken at the season's start**. It is — `seasonSeed`'s first argument and `index.ts:16500`, `:17056`, `:15885` are all `sealedEveningsOf(groupId)`, i.e. `clubRecords.eveningsSealed`.

The invariant that does **not** hold is the one the fallback silently assumes: that `eveningsSealed` – `roundsAtStart` equals the season's evenings. It only equals the season's evenings *since the counter started*.

Season 1 of this club was seeded `{roundsAtStart: 7, playedRounds: 19}` on 2026-09-16T08:36:34Z — 7 from the counter, 19 from the games. Three seals later the two had moved in lockstep (10 and 22), but the *difference* was 3 and the *truth* was 22. Two shipped code paths reached for that difference:

- `endSeasonNow` called `completedRoundsOf(groupId, seasons.roundsAtStart)` with two arguments, so `playedRounds` was `undefined` and the fallback fired. Pressing "סיים עונה עכשיו" would have sealed 3 into a write-once archive for a season the card said was 22. (LEDGER `end-season-now-ignores-playedrounds`; fixed, the fix is the comment at `index.ts:17022-17034`.)
- `reopenSeason` derived `playedRounds` the same way, then — after that was fixed — from stamped games only, which gave 3 again from the other side (LEDGER `reopen-season1-drops-the-19-unstamped-evenings`).

Both fixes are in `functions/src/`. Note ROUND 4 finding 17: HEAD's committed `functions/lib/` is the compiled output of an older commit and `seasonCounters.js` is untracked, so what git records as "built" is not what runs.

The four server derivations

All four live in `functions/src/index.ts` and delegate their *rules* to `functions/src/seasonCounters.ts`, which exists precisely because they used to be four inline loops no test could reach.

Function	Line	Source	Bound	Returns on failure
<code>sealedEveningsOf</code>	15578	<code>clubRecords.eveningsSealed</code>	1 doc	0
<code>playedEveningsOfSeason</code>	15599	games query + <code>countSeasonEvenings</code>	<code>limit(300)</code>	-1 (sentinel)
<code>playedEveningsFromGames</code>	15655	games query + <code>countPlayedEvenings</code>	<code>limit(200)</code>	0
<code>completedRoundsOf</code>	15695	<code>completedRoundsFrom(...)</code>	0-1 docs	—

```
// functions/src/index.ts:15695-15708
async function completedRoundsOf(
  groupId: string, roundsAtStart?: number, playedRounds?: number,
): Promise<number> {
  if (typeof playedRounds === 'number' && playedRounds >= 0) {
    return completedRoundsFrom(0, roundsAtStart, playedRounds);
  }
  return completedRoundsFrom(await sealedEveningsOf(groupId), roundsAtStart);
}
```

Three observations a reviewer should hold on to.

1. **The three game-scans use three different windows.** The client reads 200 docs, `playedEveningsFromGames` reads 200, `playedEveningsOfSeason` reads 300. For a club past 200 terminal games — none exist yet; the largest in the database is this one at 23 — season 1's unstamped evenings truncate differently on each surface. The doc-comment above `playedEveningsOfSeason` still claims the 200 bound (ROUND 4 24).
2. **Only `playedEveningsOfSeason` distinguishes "zero" from "I could not count".** Its -1 is honoured by exactly one caller, the reopen at `index.ts:16925-16934`. `playedEveningsFromGames` returns 0 on a failed query, and 0 is indistinguishable from a brand-new club — so a transient failure during activation seeds season 1 at zero and the club's whole history is silently dropped from its first season.
3. `completedRoundsFrom(0, roundsAtStart, playedRounds)` deliberately passes 0 as the sealed count so that the fallback branch is unreachable when the mirror is present; the `clubRecords` read is skipped entirely. Good, but it means the two branches of one function are fed by different arities at eight call sites, which is how the two-argument `endSeasonNow` bug survived review.

The frozen fourth and fifth answers

When a season closes, its evening count is copied twice and never recomputed:

- `seasonSummary/{groupId}__{seasonId}.completedRounds` — the archive, written with `create()` (`functions/src/seasonRollover.ts:584`), read by the server.
- `seasonCards/{groupId}__{seasonId}.completedRounds` — the list row, read by `src/services/seasonHistoryService.ts:82` and rendered by `SeasonHistoryScreen` , `SeasonPoster.tsx:201` and the hero tile above.

For a **running** season the same field is synthesised from counter 2 —

`src/services/seasonSummaryService.ts:512` , `completedRounds: num(seasons.playedRounds)` . So `SeasonSummaryScreen` prints `he.seasonClubRounds(model.completedRounds)` (" {n} מחזורים שוחקו בעונה הזאת " — "{n} evenings were played in the club this season") from counter 2 while the season runs and from the frozen archive afterwards. Those are different numbers with the same name, and nothing recomputes the frozen one if the counter it was copied from was wrong.

The matrix — מועדון שכחת שושי, today

Group `HhzIwmjMl1i5HS0GHt3p`. Live `seasons` block: `{enabled:true, currentId:'s2', currentNo:2, count:1, roundsAtStart:10, playedRounds:0, startedAt:1789679141952 (17.09 21:05Z), reopenedAt:1789727847680 (18.09 10:37Z), cadence:{type:'rounds', targetRounds:24}}`. `clubRecords.eveningsSealed:10`. 23 terminal games, all `finished`, of which by `eveningPlayState: 22` happened , 1 `notHappened` (`DTNscolR...`, auto-closed then `playVerified:false`). Stamps: 19 **carry none**, 3 **carry s1**, 1 **carries s3** — and `s3` does not exist.

Question asked of	Expression	Answer today	Why
Counter 1	<code>clubRecords.eveningsSealed</code>	10	blind to the 12 evenings before 26.08
Counter 2 (the card)	<code>seasons.playedRounds</code>	0	zeroed when s2 re-opened; no seal since
Counter 3, season scope	<code>getCommunityStats(g, {s2,2}).totalFinished</code>	0	no game carries an <code>s2</code> stamp, and <code>currentNo</code> is 2 so unstamped games are not admitted
Counter 3, lifetime	<code>... lifetime.totalFinished</code>	22	22 happened in the 200-doc window
Counter 3, org. rate (lifetime)	<code>lifeFinished/(lifeFinished+lifeCancelled)</code>	22/23 = 95.7%	the <code>notHappened</code> evening is the denominator's other half
<code>sealedEveningsOf</code>	—	10	
<code>playedEveningsOfSeason(g, 's2', 2)</code>	<code>countSeasonEvenings</code>	0	

Question asked of	Expression	Answer today	Why
playedEveningsOfSeason(g, 's1', 1)	countSeasonEvenings	22	19 unstamped + 3 stamped s1 , all happened
playedEveningsFromGames(g)	countPlayedEvenings	22	
completedRoundsOf(g, 10, 0) (3-arg, every close)	mirror	0	
completedRoundsOf(g, 10) (2-arg fallback)	10 - 10	0	agrees by coincidence: no seal since s2 opened
seasonCards...__s1.completedRounds	frozen	22	
seasonSummary...__s1.completedRounds	frozen	22, with roundsAtStart0 ofSeason: 7	
seasonSummary...__s1.players[*].games	rollup	max 19	three evenings never credited attendance
seasonSummary...__s1.totals.rounds	mini-games	37	the other unit, same screen family

Read that matrix as a reviewer: the club card says 0 מחזורים 24 מתוך, the stats screen's "מחזורים" tile on the *current* chip says 0, on the *all-time* chip says 22, on the 1 עונה chip says 22, and the archive of season 1 simultaneously says 22 evenings, 19 games-attended for its best attendee, and 37 משחקונים. Every one of those is produced by a different expression and only two of them share a source.

The 2-argument fallback reading 0 today is worth dwelling on: it is *right by accident*. It was 3 on 17.09 (10 - 7) and will be wrong again the moment eveningsSealed moves while roundsAtStart does not — i.e. on the next evening this club plays.

One more live consequence: roundsAtStart (10) === eveningsSealed (10) means the next seal computes seasonEvenings = eveningsSealed + 1 - tableZeroedAt = 1 (index.ts:5157), which is the "first evening of a season" case (functions/src/roundSummary.ts:698). No production roundSummaries document carries seasonEvenings at all — the field post-dates the three seals of 16–17.09 — so that suppression has never actually fired in production (ROUND 4 29, and LEDGER's rank-movement findings).

src/utils/eveningPlayed.ts — the one thing allowed to say "it happened"

248 lines, mirrored byte-for-byte into functions/src/eveningPlayed.ts below a 15-line header; tests/logic/eveningPlayedMirror.test.ts fails if they drift (though see LEDGER mirror-each-ignores-its-own-params : the describe.each declares five module pairs and the body hard-codes this one, so the other four mirrors are not actually compared).

Four states (src/utils/eveningPlayed.ts:70–71):

State	Meaning	Counts?
happened	it took place	yes, everywhere
notHappened	cancelled, or an admin said so	no — counts as a cancellation in the organisation rate
unverified	the sweep closed it, it left no trace, nobody has said	neither — excluded from both halves of the rate
pending	not over yet	nothing to decide

Three fields decide it: `status`, `endedBy` (`'admin'` | `'auto'`), and `playVerified` (`boolean`, set only by an admin resolving an unverified evening). Everything else feeds `playEvidence`.

```
// src/utils/eveningPlayed.ts:186–226 (trimmed)
export function eveningPlayStateWithReason(game) {
  if (!game) return { state: 'pending', reason: null };
  if (game.status === 'cancelled') return { state: 'notHappened', reason: null };
  if (game.status !== 'finished') return { state: 'pending', reason: null };
  if (game.playVerified === true) return { state: 'happened', reason: 'adminVerified' };
  if (game.playVerified === false) return { state: 'notHappened', reason: null };
  if (game.endedBy === 'admin') return { state: 'happened', reason: 'manualCompletion' };
  const evidence = playEvidence(game);
  if (evidence) return { state: 'happened', reason: evidence };
  if (game.endedBy === 'auto') return { state: 'unverified', reason: null };
  if (isLegacyClose(game)) return { state: 'happened', reason: 'legacy' };
  return { state: 'unverified', reason: null };
}
```

`playEvidence` (:120–172) is an ordered list of signals, most direct first: the kickoff stamp or a live `phase`; closed timer windows (`activeIntervals`); any timer press (`timerEvents`, `timerAccumulatedMs`, `timerLastStartedAt`); a server-aggregated mini-game (`committedRoundCount`); uncommitted scoreboard goals or a positive score; a committed `rotation`. There is deliberately no minimum duration — "a night that ran four minutes is still a night". A timer-only club can only ever produce the `timer` signals, and that is not a branch, it is what the list evaluates to when the other fields cannot exist.

Why the absence of `endedBy` is load-bearing. `isLegacyClose` (:181–183) is `game.endedBy !== 'admin' && game.endedBy !== 'auto'`. Every evening closed before this module shipped carries no `endedBy`, so it falls through to `happened` with reason `'legacy'` and counts exactly as it always did. That is the entire backward-compatibility story: no migration, no backfill, and not one historical number moves. On this club it is 19 of the 22 evenings — including the three with no timer, no rotation and no goals (06.07, 16.07, 04.08), which have *no evidence whatsoever* and are `happened` purely because nobody recorded how they ended. Flip the default and the club loses three evenings and its season-1 archive becomes uncomputable.

The corollary is the sharp edge: `endedBy: 'auto'` with no evidence is the *only* route to `unverified`. Once the sweep started stamping `endedBy`, the system began declining to guess — and `DTNsc01R...` is the first such evening in the database.

Why nothing may re-derive it. Before this module there were two answers: the server credited attendance on `liveMatch.startedAt` alone, the client asked only `status === 'finished'`. A night the server refused to count appeared in players' totals and in the club's organisation rate. Today the callers are `gameService.getCommunityStats ( :1156 , :1179 )`, the game list `( :327 , :1703 )`, the attendance scan `( :867 , :2107 )`, `countSeasonEvenings` and `countPlayedEvenings` `( functions/src/seasonCounters.ts:78 , :86 )`, and the seal trigger's transition gate `( index.ts:5868–5872 , !wasHappened && isHappened )`. The rule that matters for a reviewer: **if any surface re-derives "did this evening happen" from a timer field, a goal array or a status string, the three counters can no longer be reconciled**, because the arbiter would no longer be single.

The `seasonId` stamp

Who writes it: exactly one place, `functions/src/index.ts:5802–5850`, inside `onGameRosterChanged`.

```
const stampStatusChanged = before?.status !== after.status;
const stampRetryPending = (after as {seasonStampRetry?: boolean}).seasonStampRetry === true;
if (after.groupId && (stampStatusChanged || stampRetryPending) &&
    !(after as {seasonId?: string}).seasonId &&
    (after.status === 'active' || after.status === 'finished')) {
  const gSnap = await groupOnce();
  const seasons = gSnap.data()?.seasons;
  if (seasons?.enabled && seasons.currentId) {
    await event.data!.after.ref.update({ seasonId: seasons.currentId, ...clearRetry });
    stampedSeasonId = seasons.currentId;
  }
}
```

When: on the transition into `active` (where it belongs) or into `finished` (the backstop), and on any later write while the `seasonStampRetry` marker is set. **Write-once from the server:** the `!after.seasonId` guard means the stamp is never corrected once present. **Not write-once from the client:** `firestore.rules` never mentions `seasonId` on `/games` — grep it, there are zero hits outside the `seasonCards / seasonSummary` document-id comments — so a member may create a game pre-stamped with any season string and the server will not overwrite it (LEDGER `games-seasonid-unpinned`, emulator-proven with four successful writes).

What an unstamped game means: season 1. This is one rule with two copies:

```
// src/utils/seasonScope.ts:28-34
export function inSeason(game: { seasonId?: string }, season?: SeasonScope): boolean {
  if (!season) return true; // a club with no seasons: everything is in scope
  const stamp = typeof game.seasonId === 'string' ? game.seasonId : '';
  return stamp ? stamp === season.currentId : season.currentNo === 1;
}
```

```
// functions/src/seasonCounters.ts:53-60
export function eveningInSeason(game: StampedEvening, seasonId: string, seasonNo?: number): boolean {
  const stamp = typeof game.seasonId === 'string' ? game.seasonId : '';
  return stamp ? stamp === seasonId : seasonNo === 1;
}
```

They disagreed by nineteen evenings until 18.09: the client admitted unstamped games into season 1, the server did not (LEDGER `unstamped-season-1-rule-disagrees-with-the-server-by-19`).

What an orphaned stamp means: nothing, and it is unrepairable. `games/DTNscolRojYDf0ZmT0sF` carries `seasonId: 's3'` on a club whose `seasons` block knows only `s2` and whose archive holds only `s1`. It got there because season 3 briefly existed and a reopen at 18.09 10:37:27Z rolled the club back to `s2`, deleting the `s3` archive and leaving the stamp. Today that evening belongs to no scope: it is excluded from `s2` by the stamp and from `s1` by having one. It cannot be repaired — the stamp trigger only writes when the field is absent, finished games are client-read-only, and no admin tool exists. And when `s2` eventually closes, `s3` is re-issued with the same id, at which point a night played on 17.09.2026 will be inside the brand-new season 3 from its first hour (LEDGER `seasonid-stamp-orphan-unrepairable`, `orphan-season-stamp-absorbed-by-the-next-season-of-the-same-id`). It is currently harmless to the counters only because it is also `notHappened`.

Two further consequences of the write-once + transition-gated design. For the 191 clubs with seasons off, the stamp is never written, so `!after.seasonId` stays true forever and, before the transition gate was added, the group document was re-read on every write to an active or finished game (LEDGER `group-doc-read-three-times-per-sealed-evening`). And with the gate added, a game that loses its `groupOnce()` read on *both* transitions has no further transitions to retry on — hence the `seasonStampRetry` marker, which is itself defeated by the memoised-rejection bug in `groupOnce()` (ROUND 4 **22**, **23**).

What is and is not reconciled

`tests/logic/seasonCounterReconciliation.test.ts` (228 lines, added 18.09) is the assertion that was missing — LEDGER `no-test-reconciles-the-three-counters`, which is now **partially** discharged. It runs `inSeason`, `eveningInSeason`, `countSeasonEvenings`, `countPlayedEvenings` and `completedRoundsFrom` over `tests/fixtures/realClub.ts` — a fixture built from this club's production shape (22 evenings that happened, 19 unstamped, one `unverified`, one cancelled, `eveningsSealed: 10` against `roundsAtStart: 7`) — and asserts all five return 22. It passes.

What it does **not** cover, and what the reviewer should treat as still unreconciled:

- The three *writers* are untested. The `increment(1)` at `index.ts:5194` and `:5228` are checked only by `tests/logic/sealedCounterAtomicity.test.ts`, which is a regular expression run over `index.ts` as a string.
- `gameService.getCommunityStats` itself cannot be loaded by a test — it pulls React Native in through the auth layer — so the *scan* is exercised only through its two extracted rules, never end to end.
- The 200/300-document bounds, the `-1` sentinel, and the failure paths of all three game queries are untested.
- `closeSeason`, `reopenSeason`, `runSeasonRollovers`, `performSeasonClose` and `closeSeasonIfRoundsTargetMet` have no functional test at all; their names appear in the suite only inside comments (LEDGER `nothing-tests-the-sweep`).
- No test asserts that the *frozen* copies (`seasonCards.completedRounds`, `seasonSummary.completedRounds`) equal what the live counters said at the moment of the close — which is the one comparison that would have caught both the 3-vs-22 archive bugs.

§7 names the two functions that actually write the per-player and per-pair counters this section's evenings are made of, and §5 is where the frozen fourth and fifth answers are sealed.

Finally, a naming hazard the reviewer will hit within an hour: `playedRounds` means **evenings** on `groups.seasons` and **mini-games** in `src/utils/eveningStats.ts:84,164`, where it is "how many mini-games the player actually took the field for". Same identifier, two units, one repository.

4. Cadence — how a season knows it is over

A scan of all 195 `groups` documents found **two** clubs with a `seasons` block at all, and **both use the rounds cadence**. No club has ever run a date season. Everything in the date half of this section is therefore verified by reading and by arithmetic, not by observing it work.

The finish line is one function, asked by two callers

Every decision about whether a season is over passes through `seasonFinishLine` and `isSeasonDue` in `functions/src/seasonCounters.ts:136` and `:175`. This centralisation is recent; before it the hourly sweep and the seal-time close each classified a cadence for themselves, which is how one path came to close a season at a number the other had never computed.

```
// functions/src/seasonCounters.ts:136
export function seasonFinishLine(cadence: {
  type?: string; endsOn?: unknown; endsAt?: number | null; targetRounds?: number | null;
}): SeasonFinishLine {
  if (cadence?.type === 'rounds') {
    const target = cadence.targetRounds;
    return typeof target === 'number' && target > 0
      ? { kind: 'rounds', target }
      : { kind: 'none', cadence: 'rounds' };
  }
  if (isCalendarDate(cadence?.endsOn)) return { kind: 'date', endsOn: cadence.endsOn };
  if (typeof cadence?.endsAt === 'number' && cadence.endsAt > 0)
    return { kind: 'epoch', endsAt: cadence.endsAt };
  return { kind: 'none', cadence: cadence?.type ?? '(none)' };
}
```

Four kinds, not two: `rounds`, `date` (a calendar day), `epoch` (the legacy millisecond deadline), and `none`. `isSeasonDue` then answers each — `played >= target`, `isSeasonOver(endsOn, today)`, `now >= endsAt`, and `false`.

ROUNDS cadence

`cadence.targetRounds` is the season's **total** number of `מחזורים` (evenings/game-nights), not a remainder. It is measured against `seasons.playedRounds`, a mirror on the club document that is seeded to 0 when a season opens and incremented by one every time an evening is sealed. The read path is `completedRoundsOf` (`functions/src/index.ts:15695`), which delegates to `completedRoundsFrom` (`seasonCounters.ts:105`), quoted in full in §3: it returns the `playedRounds` mirror when the caller passes one, and otherwise falls back to `eveningsSealed - roundsAtStart`.

The fallback — `clubRecords/{groupId}.eveningsSealed - seasons.roundsAtStart` — is a subtraction of two counters from different eras. `eveningsSealed` only began accumulating on 26.08.2026. On the real club it currently reads **10**, while the club has played **22** evenings. Any caller that forgets to pass `playedRounds` therefore measures the season against a number twelve short of reality. The code comments record this having happened three separate times; `endSeasonNow ( index.ts:17030 )` carries a comment saying it would have sealed a season of three that the club had watched reach twenty-two.

Two paths notice that the target is met.

1. **On the seal.** `onGameRosterChanged` seals an evening, increments `playedRounds`, and then calls `closeSeasonIfRoundsTargetMet ( index.ts:15943 )` from `index.ts:5287`. It computes `playedNow = liveSeasons.playedRounds + 1` (the in-hand snapshot predates the batch it just committed), asks `seasonFinishLine`, returns immediately unless `line.kind === 'rounds'`, and closes if `played >= target` — subject to a 48-hour `REOPEN_GRACE_MS` immunity and to `clubIsQuiet`.
2. **The hourly sweep.** `runSeasonRollovers ( index.ts:16055 )`, run last inside `cronEvery60Min ( index.ts:13585 , schedule: 'every 60 minutes' )`. It pages 200 clubs at a time over `where('seasons.enabled','==',true)`, asks the same two functions, and closes at most `MAX_CLOSES_PER_SWEEP = 5` seasons per run.

A date season can only ever be closed by path 2. A rounds season is closed by whichever gets there first; `performSeasonClose` re-reads the club document and abandons the close if `currentId` has moved (`index.ts:15823-15834`).

The clamp at activation. When seasons are first switched on and the admin chooses to carry season 1 (which holds the club's entire history) forward under a rounds cadence, the server raises the target so that "continue" cannot mean "close immediately":

```
// functions/src/index.ts:16530
if (cadence.type === 'rounds' && closedSoFar === 0) {
  cadence.targetRounds = Math.max(cadence.targetRounds ?? 0, played + 1);
}
```

`played` here comes from `completedRoundsOf(groupId, existing?.roundsAtStart)` at `index.ts:16328` — **two** arguments, so it takes the `eveningsSealed - roundsAtStart` fallback. But `planActivation`, which validated the same activation ten lines earlier and produced the sheet the admin approved, was given `playedEveningsFromGames(groupId)` — a scan of the games themselves. These are different numbers: 10 and 22 on the one real club (defect `rounds-clamp-uses-a-different-history`). The clamp can only raise, and the plan's own rule (`historyExceedsTarget`, `seasonActivation.ts:113`) already requires `target > gamesCount`, so today the clamp is a silent no-op — unless `eveningsSealed` exceeds the games count, which happens whenever a finished game is deleted, because deletion decrements no counter. Then an admin who confirmed 24 gets `eveningsSealed + 1` and is never told.

`playedEveningsFromGames ( index.ts:15655 )` has two further teeth: `.limit(200)`, so a club with more than 200 finished-or-cancelled games undercounts its own history, and a `catch` that returns **0**. Zero is a claim, not an absence: it tells `planActivation` the club has never played, which disables every history refusal.

Production worked example. Season 1 of HhzIwmjMl1i5HS0GHt3p was archived at seasonSummary/HhzIwmjMl1i5HS0GHt3p__s1 with originalTarget.targetRounds = 22 and completedRounds = 22 — target met exactly. totals.rounds = 37 : that is משהקונים (mini-games), a different unit, and it is the number several client screens display beside the word מחזורים. The club is now in season 2 with roundsAtStart: 10, playedRounds: 0, cadence: {type:'rounds', targetRounds:24, months:null, endsAt:null, endsOn:null, startsOn:null}. completedRoundsFrom(_, 10, 0) returns 0, so the season is 0/24 and the fallback would agree (10 – 10 = 0). This is the only configuration in production where the two derivations happen to coincide.

DATE cadence: four fields for one deadline

Field	Type	Written by	Read by	Absent ⇒
months	1–24 int	enableClubSeasons :16396, updateSeasonTarget :16763, rebaseCadence :15441	rebaseCadence (to compute the next season's length); SeasonsSettings seeds its chips from it	rebaseCadence guesses from endsAt – startedAt, then defaults to 6
endsOn	'YYYY–MM–DD'	same three	seasonFinishLine first; SeasonsCard line + countdown	falls through to endsAt
endsAt	epoch ms	same three, except the carry-on branch which nulls it (:16463)	seasonFinishLine only if endsOn is absent; SeasonsCard legacy branch	falls through to none
startsOn	'YYYY–MM–DD'	same	nothing — one grep hit per write, zero reads	—

startsOn is write-only. endsAt is retained so that pre-1.1.7 clients keep rendering something. endsOn is the authority.

They do not agree. endsOn is computed as seasonEndDate(todayIn(), months) — the day *before* the anniversary. endsAt is computed as addMonthsClampedServer(now, months) — the anniversary instant itself. So endsAt lands on the **next season's first day** (endson–endsat–one–day–apart). Verified by executing both algorithms with TZ=UTC, as Cloud Functions run:

now	todayIn()	endsOn (3 months)	endsAt	endsAt as an Israel date
2026-09-19T07:00Z	2026-09-19	2026-12-18	2026-12-19T07:00Z	2026-12-19
2026-09-19T22:00Z	2026-09-20	2026-12-19	2026-12-19T22:00Z	2026-12-20
2026-08-31T07:00Z	2026-08-31	2026-11-29	2026-11-30T07:00Z	2026-11-30

The gap is one day in every case, but *which* day each field names depends on the hour the admin pressed the button, because todayIn uses Asia/Jerusalem and addMonthsClampedServer uses new Date().getDate(), which in Cloud Functions is UTC. An old client and a new client looking at the same season will disagree about its last day.

src/utils/seasonDates.ts , in full

166 lines, pure except one function. `functions/src/seasonDates.ts` is a byte-identical copy with a seven-line header (`diff` confirms: the header is the only difference), so every server line number in this file is the client's plus 7.

- `CLUB_TZ = 'Asia/Jerusalem' ( :21 )` — the club's calendar. Hard-coded; there is no per-club timezone anywhere in the schema.
- `CalendarDate` — a `'YYYY-MM-DD'` string. `isCalendarDate ( :28 )` is a regex test only: `'2026-13-45'` passes. The shape is chosen because lexicographic order is chronological order for it, which is what makes `compareDates ( :83 )` a two-character comparison.
- `daysInMonth(y, m) ( :43 )` — `new Date(Date.UTC(y, m, 0)).getUTCDate()`. Month is 1-indexed here and 0-indexed in `Date.UTC`, so passing `m` unchanged asks for "day 0 of month `m`", i.e. the last day of month `m-1` in 0-index = month `m` in 1-index. Correct, and leap years fall out for free.
- `addMonths(d, months) ( :55 )` — converts to an absolute month number, adds, converts back, and **clamps the day to the last valid day of the target month**. `31.08 + 6 = 28.02`, not `03.03`.
- `previousDay ( :64 ) / nextDay ( :73 )` — cross month and year boundaries with no special cases.
- `seasonEndDate(start, months) ( :94 )` — `previousDay(addMonths(start, months))`. The *inclusive last valid day*. This minus-one is why two consecutive seasons cannot both claim the anniversary.
- `nextSeasonStart(end) ( :102 )` — `nextDay(end)`.
- `todayIn(tz = CLUB_TZ, now = Date.now()) ( :113 )` — the only timezone-aware function. Uses `toLocaleDateString('en-CA')` because that locale formats as `YYYY-MM-DD`, with an `Intl.DateTimeFormat` fallback for runtimes that ignore the locale's ordering.
- `isSeasonOver(end, today) ( :138 )` — `compareDates(today, end) > 0`. Strictly greater: the season is valid **through** its last day.
- `monthsBetween ( :143 )` — whole months, used only to describe a legacy season's length.
- `MIN_SEASON_MONTHS = 1, MAX_SEASON_MONTHS = 24 ( :156-157 )`, enforced by `isValidSeasonMonths`.

Worked example: a 31st-of-the-month start, and the anniversary ratchet. Six-month seasons from `31.08.2026`, chaining `seasonEndDate` → `nextSeasonStart` :

Season	Runs	Boundary day-of-month
1	2026-08-31 → 2027-02-27	31
2	2027-02-28 → 2027-08-27	28
3	2027-08-28 → 2028-02-27	28
4	2028-02-28 → 2028-08-27	28
5	2028-08-28 → 2029-02-27	28

`addMonths` clamps *downwards only*, and `nextSeasonStart` then feeds the clamped day back in as the next season's start. The day-of-month is therefore **monotonically non-increasing over the club's life**: once a season boundary passes through February it can never return to the 31st. A club on one-month seasons from

31.01.2027 walks 31 → 28 in a single step and stays there: `2027-01-31-02-27` , `02-28-03-27` , `03-28-04-27` , ... The club permanently loses three days of the calendar month it chose. This is a consequence of the design, not a bug in it, but nothing in the UI says it will happen and no season summary records the drift.

Three different month arithmetics

Function	Where	Operates on	Timezone	Live?
<code>addMonths</code>	<code>seasonDates.ts:55</code>	calendar dates	none	yes
<code>addMonthsClampedServer</code>	<code>index.ts:15733</code>	epoch ms	the process's local zone = UTC	yes, writes <code>endsAt</code>
<code>addMonthsClamped</code>	<code>seasonLifecycle.ts:169</code>	epoch ms	the device's local zone	dead

`src/utils/seasonLifecycle.ts` — 177 lines containing a second `isSeasonDue` , `canCloseNow` , `roundsRemaining` , `validateTargetChange` , `continueSeasonTarget` and `addMonthsClamped` — is imported by exactly one file in the repository: `tests/logic/seasonLifecycle.test.ts` , which runs 22 tests against it. No screen, no service and no Cloud Function imports it. Its header claims it exists so that "the client (which greys out a button and explains why) and the server (which is the one that actually decides)" cannot disagree; in fact the client's copy of `isSeasonDue` never consults `endsOn` at all, so if it *were* wired up it would disagree with the server on every date season opened since calendar boundaries shipped. Similarly, `functions/src/index.ts:15729` declares `const MONTH_CHOICES = [1, 3, 6, 12]` under a nine-line comment about the harm it once did; grep finds no use of it.

`rebaseCadence` — the finish line moves at every rollover

`functions/src/index.ts:15405` . A **rounds** target survives a rollover untouched: "24" means "another 24 evenings" for every season. A **date** target cannot, because an inherited end date is already in the past and would make the new season due the instant it opened — one empty archived season per hour, for ever.

```
function rebaseCadence(cadence, seasonStartedAt, now) {
  if (!cadence || cadence.type !== 'date') {
    return (cadence ?? { type: 'date', months: 6 }) as { ... };
  }
  let months = isValidSeasonMonths(Number(cadence.months)) ? Number(cadence.months) : 0;
  if (!months && seasonStartedAt > 0 && typeof cadence.endsAt === 'number') {
    const ranMonths = Math.round((cadence.endsAt - seasonStartedAt) / (30*24*60*60*1000));
    months = isValidSeasonMonths(ranMonths) ? ranMonths : 0;
  }
  if (!months) months = 6;
  const startsOn = todayIn(undefined, now);
  return { type: 'date', months, endsAt: addMonthsClampedServer(now, months),
    startsOn, endsOn: seasonEndDate(startsOn, months), targetRounds: null };
}
```

Caller	Line	What it is rebasing
performSeasonClose	15886	the successor of a season the sweep or the seal just closed
enableClubSeasons , seal-now branch	16505	season $N+1$, measured from the activation moment
reopenLastSeason	16957	the season being brought back
endSeasonNow	17061	the successor of a manually-ended season

Two problems live here.

A missing cadence produces a season that can never end. The first branch returns `{type: 'date', months: 6}` with **no** `endsOn` **and no** `endsAt`. Fed to `seasonFinishLine`, that is `{kind: 'none', cadence: 'date'}` — and `isSeasonDue` returns `false` for `none`, for ever. Any close of a season whose club document lacks a `cadence` map opens a successor that no path will ever close. This is `cadence-with-no-finish-line-never-closes`. The sweep at least now *says so* (`index.ts:16141-16152`, `noFinishLine += 1` plus a `console.warn`) instead of the bare `continue` it used to be — but it is a log line, not an error row, and the production `errors` collection has never held a seasons entry.

Reopen gives a date season a whole new life. `reopenLastSeason` is the undo button for an accidental close. It rebases from `archive.originalTarget` — so a six-month season that was ended three days early comes back running **six months from today**, not three days (`reopen-extends-a-date-season-by-a-full-length`). `rebaseCadence` is the function for opening the *next* season; the reopen needs the original `endsOn` restored, and there is no code that does that.

The four end-date options

On a **first** activation with a date cadence and "carry season 1 on", the admin must name season 1's last day, because season 1 contains the club's entire history and has no computable start. `SeasonsSettings.tsx:67` defines the shortcuts and `:831-845` renders them as dates:

```

const SEASON1_MONTH_CHOICES = [1, 2, 3, 6] as const;
...
{SEASON1_MONTH_CHOICES.map((m) => {
  const d = seasonEndDate(today, m);
  return <Chip key={m} label={formatCalendarDate(d)} active={!season1Custom && season1EndsOn === (
    onPress={() => { setSeason1Custom(false); setSeason1Months(m); setSeason1EndsOn(d); }}
  )}}

```

They are deliberately shorter than the season-length chips (`MONTH_CHOICES = [3, 6, 12]` , `:62`) because this season already holds everything played so far. A fifth chip, "מותאם אישית" ("custom"), opens a 1–24 stepper that also writes a date rather than a number — the comment at `:867` records that it previously moved a number while the date the admin was actually choosing never changed.

Verified against `today = 2026-09-19` : the chips render 18.10.2026, 18.11.2026, 18.12.2026 and 18.03.2027, and `nextSeasonStart` of each is the 19th. Correct, and consistent with `planActivation`'s `nextStartsOn` . `planActivation` (`seasonActivation.ts:174`) then refuses `end <= today` — so the same-day option, if a custom stepper could reach it, would be rejected with `season1EndNotFuture` → "חייב להיות N תאריך הסיום של עונה" ("season N's end date must be in the future").

Three ranges are in play for one concept and they do not match: the client offers 3/6/12, the client's custom stepper allows 1–24, the server accepts 1–24 (`isValidSeasonMonths`), and the dead server constant says 1/3/6/12. The server's date branch also still silently rewrites anything outside 1–24 — including a missing `months` field — to 6, directly under a comment declaring that "rewriting silently is not [honest], and 6 was neither what they picked nor what they confirmed". The class of input was narrowed, not eliminated.

updateSeasonTarget — moving the line mid-season

`functions/src/index.ts:16627` . The client calls it from `SeasonsSettings.saveTarget` (`:513`) with `{groupId, cadenceType, months?, targetRounds?}` and nothing else.

What an admin can change: the target number, and the cadence *type* itself — rounds↔date, mid-season, with no extra guard. A club twelve evenings into a 24-evening season can switch to a three-month date cadence; `playedRounds` stays at 12 but stops meaning anything.

What is refused:

Condition	Code	Message the admin sees
not an admin	<code>permission-denied</code>	"הפעולה מיועדת למנהלי המועדון"
<code>seasons.enabled !== true</code>	<code>failed-precondition</code> <code>seasons are off</code>	mapped
<code>rounds</code> , non-finite or ≤ 0	<code>invalid-argument</code> <code>targetRounds required</code>	mapped → <code>targetInvalid</code>

Condition	Code	Message the admin sees
rounds < MIN_SEASON_ROUND S (2), :16723	invalid-argument target N is below the 2-round floor	unmapped — refusalOf (seasonService.ts:145) only matches invalid-argument messages containing the literal targetRounds , so this reaches the admin as the generic "משהו השתבש"
rounds <= played , :16730	failed-precondition target N is not above the M rounds already played	mapped → targetBehind , and playedFromError recovers M
rounds <= played and an evening in play :16686	failed-precondition openGame / unsealedGame	mapped

Nothing is refused on the date side. months is at least 1 by the time it is used, so addMonthsClampedServer(now, months) is always in the future: **a date re-target can only ever extend a running season**, never shorten it below one month from today, and never close it.

What targetHistory records. One arrayUnion entry per accepted change (:16802): at , by , byName (frozen), from (the whole previous cadence map, or null), to (the whole new one), playedAtMove (sealed because played is a moving number), and endsTheSeason .

What it does not record. No seasonId — which is exactly what let a reopen reattach one season's history to another (target-history-entry-carries-no-season-identity). Not the initial target: the first entry's from is the only trace of it. Not the rewrites rebaseCadence performs at every rollover and at every reopen, which move a date club's finish line with no entry at all. And nothing shows it: targetHistory is typed (src/types/index.ts:1004), deserialized (src/firebase/firestore.ts:691) and unit-tested (tests/logic/groupSeasonsReader.test.ts:50), and rendered by no screen — there is no Hebrew string for it (target-history-never-shown-to-anyone). The client's SeasonTargetChange type also describes only {type, endsAt, targetRounds} for from/to, so months , endsOn , startsOn , playedAtMove and endsTheSeason are invisible to TypeScript even though they are in the document.

target-history-erased-at-close . performSeasonClose (:15911) and endSeasonNow (:17062) both write targetHistory: [] for the successor season. The array is now sealed into the archive first (:15838 , :17042), but the one archive that exists in production — seasonSummary/HhzIwmjMl1i5HS0GHt3p__s1 , whose full key set is awards, closedAt, completedRounds, endsAt, groupId, groupName, no, originalTarget, pairs, players, roundsAtStartOfSeason, seasonId, startsAt, totals, zeroedAt — **has no targetHistory key**. It was closed before the sealing existed. That club's finish line moved from 22 to 2 to 24 and only the last move survives anywhere.

target-change-can-close-a-season-with-no-confirmation is now half fixed, and the fix made a second mechanism unreachable. The asked <= played throw at :16730 means the endsTheSeason computation twenty lines later —

```
const endsTheSeason = next.type === 'rounds'
  && typeof next.targetRounds === 'number' && next.targetRounds <= played;
```

— **can never be true**, because that call has already thrown. Consequently `seasons.targetMovedToClose` is written by nothing (grep: one gated write at :16816 , three unconditional deletes, one read), and the `endedEarly / closedBy` branch in `performSeasonClose` at :15845–15864 that exists to read it back is dead. The residual hole is `played + 1`: it is accepted, it is not flagged, and the very next seal closes the season — from a plain one-tap button, because `SeasonsSettings` routes a live club straight to the server (:953 : `onPress={live ? saveTarget : () => setConfirmOpen(true)}`) while the functionally identical "סיים עונה" ("end the season now") gets an `appAlert` confirmation. The code comment at :16787 argues `played + 1` is a season "reaching its finish line", which is defensible for the flag and not for the missing confirmation.

`date-retarget-restarts-the-clock`. The date branch writes `startsOn: todayIn(now)` and recomputes `endsOn` from today, while `seasons.startedAt` keeps the season's real start. The same season then has two start dates: `SeasonsCard`'s progress bar uses `seasons.startedAt` (`SeasonsCard.tsx:123`) and the settings preview labels today "תחילת העונה" ("start of the season", `he.ts:2310`). `months` means "length" when a season is enabled and "time remaining from now" when it is re-targeted, with no word anywhere marking the difference.

`date-season-cannot-be-renewed-at-the-same-length`. The save button is gated on `targetChanged` (`SeasonsSettings.tsx:401`):

```
if (cadence === 'rounds') return c?.type !== 'rounds' || c.targetRounds !== rounds;
return c?.type !== 'date' || c.months !== months;
```

For a date club the comparison is on `months` alone — the very field the server does *not* treat as an identity. An admin who wants "another three months from today", which is precisely the move the 48-hour reopen grace exists to allow, finds the button disabled and the line "זה היעד שמוגדר כרגע במועדון" ("this is the club's current target"). The one correct action is the one the UI calls a no-op.

Finally, `SeasonsSettings` seeds `history` for a live club from `seasons.playedRounds` (:336), which is the same figure the server measures against — so `settings-screen-validates-a-live-target-against-zero` is fixed at HEAD. Per ROUND4 finding 1, neither that fix nor any other client fix from 18.09 is in the shipped store binaries (see *What is running where*, front matter).

5. Closing a season

Everything in this section is quoted from `HEAD = e35394a`, which for `functions/` is what production runs: `git diff --stat 8235851 HEAD -- functions/` is empty and every seasons function was redeployed at 18:16Z on 2026-09-18, two and a half minutes after that commit (front matter, *What is running where*). Nothing in this section is a fix waiting on a deploy; where a fix landed after the one real close ran, that is said, because the *data* the older code wrote is still there.

Three doors into one room

A season can be closed by exactly three pieces of code. All three end in the same function, `closeSeason` in `functions/src/seasonRollover.ts:307`, and two of the three first pass through `performSeasonClose` (`functions/src/index.ts:15790`), which wraps the close in a pre-flight check and a lifecycle write.

	<code>closeSeasonIfRoundsTargetMet</code>	<code>runSeasonRollovers</code>	<code>endSeasonNow</code>
where	<code>index.ts:15943</code>	<code>index.ts:16055</code>	<code>index.ts:16971</code>
fires on	the seal of an evening, from <code>onGameRosterChanged</code> (call site <code>index.ts:5274–5292</code>)	<code>cronEvery60Min</code> , last in the job (<code>index.ts:13604</code>)	an admin tapping סיים עונה עכשיו ("end season now")
cadences it can finish	<code>rounds</code> only (<code>line.kind !== 'rounds' → return, :15967</code>)	all of them	ignores the cadence entirely
quiet check	<code>clubIsQuiet(groupId, {mode: 'afterSeal', exceptGameId})</code>	<code>clubIsQuiet(doc.id)</code> (sweep mode)	<code>clubIsQuiet(groupId)</code> (sweep mode)
on "busy"	logs, returns; the sweep is the backstop	stamps <code>seasons.dueBlockedSince</code> , retries next hour	throws <code>failed-precondition</code> to the client
goes via <code>performSeasonClose</code>	yes	yes	no — it calls <code>closeSeason</code> directly and writes its own lifecycle block
extra archive fields	<code>endedEarly</code> / <code>closedBy</code> only if <code>targetMovedToClose</code> is set	same	always <code>endedEarly:true</code> , <code>closedBy</code> , <code>closedByName</code>
<code>completedRounds</code>	<code>known.played = mirror + 1</code>	<code>completedRoundsOf(id, roundsAtStart, playedRounds)</code>	<code>completedRoundsOf(id, roundsAtStart, playedRounds)</code>

The last row is the one that used to differ, and it is worth being exact about, because whatever it produces is written into a document created with `create()` and never recomputed.

`completedRounds` : the argument the three paths disagreed about

There are four numbers in this system that all answer "how many `מהזורים` (evenings) has this season held", and they are not equal. All three close paths route through one function, `completedRoundsFrom` (`functions/src/seasonCounters.ts:105` , quoted in full in §3): it returns the `playedRounds` mirror when the caller passes one, and otherwise subtracts `roundsAtStart` from `clubRecords.eveningsSealed` .

`playedRounds` is the **mirror**: seeded when the season opens (`functions/src/seasonSeed.ts:26`) from `playedEveningsFromGames` — a `didEveningHappen` count over the club's last 200 terminal games — and incremented by one on every seal. `roundsAtStart` is `clubRecords.eveningsSealed` at the moment the season opened. The fallback branch, `eveningsSealed - roundsAtStart` , subtracts two counters from different eras: `eveningsSealed` only began being written on 26.08.2026, and the club's games go back to June.

On `HhzIwmjMl1i5HS0GHt3p` (`מועדון שכחת שושי`, the only club that has ever run a season) at the moment season 1 closed — 2026-09-17 16:12:06Z — the live values were `roundsAtStart: 7` , `clubRecords.eveningsSealed: 10` , `seasons.playedRounds: 21` (the seal that triggered the close had just made it 22). So:

path	expression	value
seal	<code>liveSeasons.playedRounds + 1 (index.ts:5282-5285)</code>	22
sweep	<code>completedRoundsFrom(_, 7, 22)</code>	22
<code>endSeasonNow</code> , as deployed before 18.09	<code>completedRoundsFrom(10, 7, undefined)</code>	3

That third row is finding **endseasonnow-archives-wrong-length** (P0, confirmed twice). The callable simply omitted the third argument; `seasons.playedRounds` was not even in its local type literal. Had the admin pressed the button instead of playing the twenty-second evening, the club would hold a permanent archive saying season 1 lasted three evenings. It is fixed at `index.ts:17030-17034` and the comment there records the counterfactual. The fix is deployed.

The sealed archive proves the surviving path was correct: `seasonSummary/HhzIwmjMl1i5HS0GHt3p__s1` carries `completedRounds: 22` .

`closeSeason` , in order

WHAT IT READS

```
// seasonRollover.ts:310
const [psSnap, csSnap, pairSnap] = await Promise.all([
  db.collection('communityPlayerStats').where('groupId', '=', groupId).get(),
  db.collection('communityStats').doc(groupId).get(),
  db.collection('communityPairStats').where('groupId', '=', groupId).get(),
]);
```

Three reads, then a fourth: one `getAll` per 300 members over `/users`, because `communityPlayerStats` carries no name and the archive must still render after an account is deleted (:336–348). The pair query is the expensive one — see the guest-pair discussion below.

THE PLAYERS MAP

```
// seasonRollover.ts:366
if (num(x.rounds) === 0 && num(x.games) === 0) continue;
```

A row with no mini-games **and** no evenings is not a participant. This matters more than it looks: nothing ever deletes a `communityPlayerStats` row, and the close winds rows back to zero rather than deleting them, so after one close every member of the club is sitting at zero. Without this test every subsequent season's archive would carry every ex-member forever. It also means a close run over a table that a previous close already emptied archives `players: {}` — which is exactly what happened on this club four hours later; see the worked example.

Two fields are deliberately allowed to be **absent** rather than zero (:377–379): `csRounds` and `asRounds`, the coverage denominators for clean-sheet and assist rates. They arrived later than the metrics they divide, and writing `0` would say "measured over zero rounds", which readers cannot tell from a real zero.

The field list is `PLAYER_SEASON_FIELDS` (:184–211) — 19 counters including `eveningScoreSum` and `eveningScoreCount`, which must reset together or the MVP average stops being an average. Everything not on that list survives the close: `bestEvening` (a personal high-water mark), `lastEveningScore`, and `kingGoalsSum / kingGoalsCount` (the benchmark the evening score is graded against). The header comment at :18–24 explains why a blanket overwrite here would silently inflate every future evening score by about a quarter of the scale.

THE TOTALS

`CLUB_SEASON_FIELDS` (:234–242) is copied off `communityStats`; `assists` and `cleanSheets` have no club counter at all, so they are summed from the member rows (:392–399) — otherwise they become underivable the moment the rows are zeroed.

THE PAIRS, AND THE CAP

Three filters, in order (:417–479):

```
if (!isReal(a) || !isReal(b)) { droppedGuestPairs += 1; continue; } // :436
const playedTogether = num(x.sameTeam) + num(x.against) + num(x.assists)
  + num(x.assistsAToB) + num(x.assistsBToA);
if (playedTogether === 0) { droppedEmptyPairs += 1; continue; } // :450
if (Object.keys(pairs).length >= MAX_ARCHIVED_PAIRS) { droppedOverflowPairs += 1; continue; }
```

`isReal` is `!id.startsWith('guest:')` (:31). `MAX_ARCHIVED_PAIRS` is 1,200 (:181), sized so the archive cannot exceed Firestore's 1 MB document limit and make the club *permanently unable to close a season* — the failure mode is unrecoverable, because a season summary is written with `create()`.

Guests are the reason the cap exists. A guest identity is minted fresh for every game, so `communityPairStats` gains a permanent new document for every stranger who ever turns out. Production, counted by paging the whole collection today: **314 pair documents for a seven-player club, 293 of them guest pairs (93%), 21 real**. The collection holds 717 documents across the entire database, so this one club is 44% of it. Finding **guest-pairs-make-every-close-read-fifteen-times-what-it-archives** (P2) is exactly this: the close reads 314 documents to archive at most 21, the sweep runs last inside `cronEvery60Min` 's shared 540-second budget, and at roughly 45 pair documents per player a sixty-player club is about 2,700 reads per close. The wipe now deletes guest pair rows outright (:944–948 , landed in 5531a21 and deployed), which stops the *growth* — but no close has run since, so the 293 documents are still there, and the read amplification is paid in full on the next one.

The surviving pair is stored under a sorted key, with `a / b` and the four directional counters flipped to match (`seasonPairKey` , :104–110). One key, derived one way, used by both the wipe and the restore: if they disagreed a reopen would give a pair chemistry it never had.

THE ARCHIVE WRITE, AND THE `CREATE()` LATCH

```
// seasonRollover.ts:580
const summaryRef = db.collection('seasonSummary').doc(`${groupId}__${seasonId}`);
try {
  await summaryRef.create({ /* ... totals, players, pairs, awards ... */ });
} catch (err) {
  const code = (err as { code?: number | string }).code;
  if (code !== 6 && code !== 'already-exists') throw err;
  const existing = await summaryRef.get();
  if (existing.get('zeroedAt')) { // :667
    console.log('[season] already closed – skip', groupId, seasonId);
    return { archived: false, players: 0, pairs: 0 };
  }
  resuming = true; // :676
  resumedAwards = existing.get('awards');
  resumedPlayers = existing.get('players');
  resumedTotals = existing.get('totals');
  resumedEvenings = existing.get('completedRounds');
}
```

That single `create()` is the whole idempotency story, and the ordering — build in memory, archive, *then* wipe — is what stops a retry from sealing a season of zeros over the real one.

`zeroedAt` (written last, :1024) separates the two meanings of `ALREADY_EXISTS`. Present: the first pass finished, the live rows now belong to the next season, stop. Absent: the first pass died between the archive landing and the wipe, and the club is half-closed — the season sealed, the titles never written, the table never reset. On the resume path every derived value is read back off the archive, not recomputed, because the live rows may be half-wiped.

[defect: resume-subtracts-recomputed-row, P2] — with one exception, and it is the whole of the wind-back. `resumedPlayers` is used for the card (:744) and `resumedAwards` for the card and the titles (:717 , :832), but the subtraction at :879 and :903 uses `players` — the map recomputed from the live rows at the top of *this* invocation:

```
// seasonRollover.ts:878
for (const d of psSnap.docs) {
  const archivedRow = players[(d.data() as { userId?: string }).userId ?? ''];
  if (!archivedRow) continue;
  ...
  Object.assign(patch, windBackRow(data, archivedRow, PLAYER_SEASON_FIELDS));
}
```

On a resume, an evening played between the crash and the retry is inside `players` but not inside the sealed archive. Subtracting the recomputed row therefore removes that evening from the live table while the archive never recorded it: it exists nowhere. This is the precise failure the module header at :46–61 says subtraction exists to prevent, on the one path where the two rows differ.

THE CARD

`seasonCards/{groupId}__{seasonId}` (:755–809) is a compact projection for the hall of fame — date, three totals, a participant count and the resolved winner names — because the archive is large and the list screen would otherwise pull every season in full. Two details:

- `countSeasonParticipants` (`functions/src/seasonParticipants.ts:23`) counts `games > 0 || rounds > 0` . It shipped counting `rounds` alone, which is **משחקונים** (mini-games), which only exist in advanced mode — so every season a timer-only club closed reported "0 שחקנים" ("0 players") no matter how many people turned up.
- `sealedCardEvenings` (:155–165) prefers the archived `completedRounds` , then the caller's, then the awards denominator. The card prints this as "מחזוריים". It is written with `merge:true` , so a resume converges rather than duplicating.

The field `awardsDenominator` lived on both the archive and the card for one day and was removed in 8235851. `awardsDenominatorOf` (:133–139) is `max(players.games)` , and `mostLoyal` is the maximum of the same array, so `winner.value / awardsDenominator` is 1.0 by construction — the medal tier it fed was platinum for every season of every club, forever. Production proves the identity: s1's archived `games` are {19,18,18,18,18,18,17} and `awards.mostLoyal.value` is 19. The number survives only as the awards *eligibility gate's* denominator (:556–559), never as a published share.

THE TITLES

:823–860 writes one document per winner per title at `users/{uid}/seasonTitles/{groupId}__{seasonId}__{titleKey}` , with `groupName` frozen, batched 400 at a time, outside the `create()` latch on purpose. A pair title's key is split on `__` so both halves get a copy; `guest:` ids are skipped. `if (!award) continue` — a null award is a result, not a gap. Production holds 14 such documents for s1: eight titles, of which `mvp` has seven winners and `deadlyDuo` is null.

THE WIND-BACK

The invariant, stated at :46–61 and implemented in `windBackRow` (:62–74):

```
out[f] = archived ? Math.max(0, rowNum(current?.[f]) - rowNum(archived[f])) : 0;
```

Subtract what was archived; do not write zeroes. The rows were read at the top and are wiped at the bottom; anything committed in between is not in the archive, and an absolute zero would erase it from the live table too, leaving it nowhere. `archived === null` means "this row belongs to no season" — a pair past the cap — and is zeroed. Subtraction is not idempotent, so each row gets `seasonWoundBack: seasonId` and `seasonReopened` is deleted so a later reopen can stamp its own restore. Player rows go one transaction each (:881); pair rows go in 400-op batches (:980).

The skip guards read `if (resuming && row.seasonWoundBack === seasonId)` (:886 for players, :973 for pairs). The `resuming &&` half is the whole guard. Without it, a stamp naming the season being closed was treated as proof that season had already been wound back — but a *reopen* leaves that stamp on every row the close touched and the restore did not reach, so the next close would archive that chemistry *and* skip wiping it, sealing it into a write-once archive while leaving it standing live for the successor to inherit. That is finding **stale-pair-woundback-stamp-skips-next-wipe** (P1), and the state it needs is on this club right now: all 314 pair documents carry `seasonWoundBack: "s2"` while **s2 is the running season**. The qualifier landed in 8235851 and is deployed, so the next close of s2 will wind those 21 real pairs back correctly. The anomalous data survives; the defect does not. The reopen also sweeps stale stamps (:1174–1198) — deployed as well, but it runs only on a reopen, and the reopen that left these stamps ran seven and a half hours before that code went out.

[defect: club-totals-zeroed-not-subtracted, P1] The club document breaks the rule the module is built on:

```
// seasonRollover.ts:1004
const zeroClub: Record<string, number> = {};
for (const f of CLUB_SEASON_FIELDS) zeroClub[f] = 0;
await db.collection('communityStats').doc(groupId).set({ ...zeroClub, ... }, { merge: true });
```

Absolute zero, not `windBackRow`. Player rows and pair rows subtract; the seven club counters (`rounds`, `goals`, `guestGoals`, `ownGoals`, `tiedRounds`, `shootoutRounds`, `scorelessRounds`) do not. An evening committed between the read at :310 and this write is preserved on every player row and destroyed on the club total, so the club's goals stop equalling the sum of its members' goals with nothing anywhere recording the discrepancy. `chemistrySince` is re-stamped to `now` in the same write, which is correct — otherwise the new season's chemistry card would date itself from the old one.

Season 1 of מועדון שחקת שושי, reconciled against the games

`seasonSummary/HhzIwmjMl1i5HSOGHt3p__s1`, read live:

field	value	reconciles to
startsAt	1789547794676 = 2026-09-16 08:36:34Z	the moment seasons were switched on , not the club's first game (June). The hall of fame therefore captions three months of football with a day and a half — "16 בספט' 17 – בספט' 16" at HEAD, "2026 ספט' – 2026 ספט'" in the shipped build (§11) — above "22 מחזורים". Finding season1-startsat-is-the-enable-moment
endsAt = closedAt	1789661526768 = 2026-09-17 16:12:06Z	the seal of TZ7IWEJWvw2itvIKCd8
zeroedAt	1789661533425	6.7 s later — the whole close took under seven seconds for 7 players and 314 pair rows
completedRounds	22	19 unstamped legacy evenings + 3 stamped s1 (yA48XsyC , nRHEgxVt , TZ7IWEJW); a 23rd finished game, DTNscolR , is stamped s3 and carries playVerified:false
roundsAtStartOfSeason	7	clubRecords.eveningsSealed on 16.09. Ten roundSummaries documents exist for this club; seven were already written when seasons were switched on, and three (yA48XsyC , nRHEgxVt , TZ7IWEJW) were sealed during s1
originalTarget	{type: 'rounds', targetRounds: 22}	the cadence at close time; the club's current cadence is 24, moved on 18.09
totals.rounds	37	משחקונים, mini-games — not evenings. The same document says 22 in completedRounds . Both numbers are correct and they measure different things
totals.goals / guestGoals	27 / 4	
totals.assists	13	= 1+5+1+2+3+0+1, summed from the seven member rows at :392
totals.cleanSheets	75	= 12+11+13+8+10+11+10, summed at :396
players	7 rows	
pairs	21	exactly C(7,2) — every member pair played together at least once; all 293 guest pairs dropped
awards	8 of 9 decided	deadlyDuo: null

The seven archived games values are {19, 18, 18, 18, 18, 18, 17} . Counted directly from the 22 game documents' players arrays, actual attendance is {22, 20, 20, 20, 20, 20, 19} . The gap is finding **archive-games-undercounts-every-player** (P1) and it resolves exactly: three evenings — 2SMrLCGH (06.07), MMtE8J6H (16.07), FCa5UtSd (04.08) — never left liveMatch.phase: "organizing", have rotation:

`null`, and credited nobody a `games` increment. `didEveningHappen` counts them (they are legacy closes with no `endedBy`, so `isLegacyClose` (`functions/src/eveningPlayed.ts:196`, `endedBy` neither `'admin'` nor `'auto'`) calls them 'happened'), the per-player counter does not. `מתן לוי` attended all three ($22 - 3 = 19$); the other five regulars attended two each ($20 - 2 = 18$); `אביציק לוי` attended two ($19 - 2 = 17$). §7 confirms the mechanism from the other end: those three games, and only those three, have an empty `finishCredited` subcollection, which is the idempotency marker written in the same batch as the `games: increment(1)`. **The archive's own season length and its own per-player attendance are counted by two different rules that disagree by three on this club** — which is precisely why `awardsDenominatorOf` exists, and why the eligibility gate is set from `max(games) = 19` rather than from `completedRounds = 22`.

The pair archive reconciles too: summed over the 21 entries, `sameTeam` 94, `against` 140, `winsTogether` 46, `lossesTogether` 35, `cleanSheetsTogether` 50, `winsA` 62, `winsB` 52, directional assists $4 + 3 = 7$ against a legacy undirected `assists` of 8. `deadlyDuo` is null because the best pair score is 2 and the floor is `MIN_DUO_ASSISTS - 1 = 2` (`seasonAwards.ts:137`, passed at `:244`) tested with a strict `best <= floor → null` at `:156`.

`mvp` is awarded to **all seven members at exactly 6.0**. Every player's `eveningScoreSum / eveningScoreCount` is $18/3$ or $12/2$. 6.0 is the bottom of the evening-score scale and also what the score returns for a player who took the field for no mini-games. `MVP_SCALE_FLOOR = 6` (`seasonAwards.ts:141`, passed at `:221`) with the same strict test at `:156` now blocks this — but s1's archive is write-once and seven junk `שחקן העונה` ("player of the season") title documents sit on seven profiles permanently. Finding **mvp-title-crowns-the-entire-club** (P1).

AND THE CLOSE THAT ARCHIVED NOTHING

Four hours after s1 closed, at 21:12:08Z, a second close ran on the same club. Its archive has since been deleted by a reopen, but the state it left behind is unambiguous and is still on disk:

- All seven `communityPlayerStats` rows read `{games:0, rounds:0, goals:0, ...}` and still carry `seasonWoundBack: "s1"` — **not** `"s2"`. They were already empty, so the `:366` filter excluded every one of them from `players`, so `players` was `{}`, so the wind-back loop's `if (!archivedRow) continue` (`:880`) skipped every row and wrote no stamp.
- All 314 pair rows read zero and carry `seasonWoundBack: "s2"` — the wipe reached them (guest deletion had not yet been written, so they were zeroed rather than removed). The counts are readable from the documents themselves: 314 rows for this club, 293 of them guest pairs, 21 real, and the deleted archive would have held none of them, because every one of the 21 real rows had already been emptied by the s1 close four hours earlier.
- `communityStats` is all zeros with `seasonReopened: "s2"`.

So the close archived an empty `players` map, an empty `pairs` map, zero totals, and — because `endSeasonNow` still passed two arguments, giving `completedRounds = 10 - 10 = 0`, and `computeSeasonAwards` returns all-null for `completedRounds <= 0` (`seasonAwards.ts:199-205`) — nine null awards. It then wrote a `seasonCards` document reading `"שחקנים 0"`, pushed nobody (`announceSeasonClosed` at `index.ts:15342-15351` iterates the empty `players` map), and advanced the club to season 3. `games/DTNscolRojYDf0ZmT0sF`, kicked off seven minutes later, still carries the orphan stamp `seasonId: "s3"` on a club whose `seasons` block now knows only s2.

This is finding **perform-close-writes-next-season-even-when-nothing-was-archived** (P1) in its observable form: *nothing in either close path is conditional on the archive containing anything.*

`clubIsQuiet`

`index.ts:15466–15571`. The one thing a close must not do is split an evening: each mini-game commits separately, so mini-games 1-3 would land in the old season and 4-6 in the new, while the player's career total — written in the same batch — keeps the whole. Two checks.

Check one: is a game in play. A `games` query filtered by status and bounded on both sides:

```
// index.ts:15495
.where('status', 'in', afterSeal ? ['active'] : ['scheduled','open','locked','active'])
.where('startsAt', '<=', now + STARTING_SOON_MS)    // +3h
.where('startsAt', '>=', now - TONIGHT_MS)          // -12h
.limit(2)
```

The upper bound is `now + 3h` because a game can be started before its scheduled kickoff. The lower bound exists because a game left open days ago is stale and must not hold a club's season hostage forever. The status list narrows to `['active']` on the `afterSeal` path: the evening the close would split just ended, so only a genuinely-in-play second game can block, not next week's recurring clone — and *nearly every real club runs a recurring fixture*, which is why the unbounded earlier version meant no such club could ever close a season on any path.

Check two: is an evening finished but not yet sealed. The three newest `finished` games; for each, if it has no `roundSummaries/{gameId}` document, block. Closing in that gap makes `sealRoundSummary` compare tonight against a table with no history, so every stat reads as a brand-new club record — and the summary is written once, so the wrong story is permanent.

Three floors let a game out of check two (`:15542` , `:15557` , `:15564`): the evening did not happen per `didEveningHappen` ; it is a **legacy close** (`endedBy` is neither `'admin'` nor `'auto'`); or it started more than `STALE_SEAL_MS` (24 h) ago.

[defect: quiet-legacy-evening-blocks-club-forever, P1-really-P0] The legacy floor at `:15557` is the fix; before it, that clause did not exist. `roundSummaries` began on 26.08.2026, so every evening finished before that date carries no `endedBy` , is called 'happened' by `isLegacyClose` , and will never be sealed by anything. "Finished but not yet sealed" was therefore *permanently* true for them: such a club could not enable seasons with "seal now", could not end a season by button or by sweep, and could not undo a close, while the Hebrew told the admin to wait for something that was never going to happen. The window only advances when the club plays a *new* evening, and a finished game is terminal.

Provenance warning. An auditor ported `clubIsQuiet` and `didEveningHappen` out of the repository, ran them over every finished game in production, and reported **24 of the 30 clubs that have ever finished a game returning** `{ok:false, blocker:'unsealedGame'}` **permanently**. That script is not in the repository, the club ids were not recorded, and nothing here reproduces it. It is the largest adoption claim in this document and the one number a reviewer cannot re-derive as written — treat it as a hypothesis with a clear

mechanism rather than as a measurement. What is independently checkable is the mechanism itself (`roundSummaries` exists for 10 of this club's 23 finished games, all after 26.08.2026) and the outcome (exactly one club in a 195-club database has ever run a season).

The fix is deployed; the historical distortion of every adoption number is not undone by it.

One more note on scope: `clubIsQuiet` is a per-club gate on *closing*, not a lock. Nothing prevents a mini-game from committing during the seven seconds the close takes; that is what the subtraction semantics exist for, and what the club-totals zero throws away.

`performSeasonClose` : the rewrite of the `seasons` block

Two things happen around `closeSeason` here.

Pre-flight (`index.ts:15823–15834`). Both callers arrive holding a snapshot that can be minutes old, so the group document is re-read:

```
const preSnap = await groupRef.get();
const pre = preSnap.data()?.seasons;
if (pre?.enabled !== true || pre?.currentId !== seasonId) {
  console.log('[season] close abandoned – the club moved under us', ...);
  return false;
}
```

[defect: sweep-closes-a-club-that-turned-seasons-off, P1] This check is the fix for that finding. The sweep takes one paged snapshot at the top of a run that walks every enabled club; a club that disabled seasons in between would previously have had its table wiped, its titles awarded, everyone pushed, and been left with seasons off and an archive no screen can reach — because the lifecycle write below does not carry `enabled`, so nothing turns them back on. Note the check is a plain read, not part of a transaction with the close: the window is narrowed from minutes to milliseconds, not eliminated.

The same read harvests two fields that the lifecycle write is about to destroy: `targetHistory` (reset to `[]` for the next season, so the archive is the only place it can survive — this club went 22 → 2 → 24 and only the last survives anywhere) and `targetMovedToClose`, which, if present, marks the archive `endedEarly` with the admin who moved the finish line onto the club's current position.

The lifecycle write (`:15887–15924`), in a transaction re-checking `currentId` :

```
tx.set(groupRef, { seasons: {
  currentNo: nextNo, currentId: `s${nextNo}`,
  startedAt: now,
  roundsAtStart: nextRoundsAtStart,    // await sealedEveningsOf(groupId) – the ABSOLUTE count
  playedRounds: 0,
  reopenedAt: 0,
  cadence: nextCadence,                // rebaseCadence(cadence, args.startedAt, now)
  targetHistory: [],
  targetMovedToClose: FieldValue.delete(),
  dueBlockedSince: FieldValue.delete(),
  count: closedSoFar + 1,
} }, { merge: true });
```

`rebaseCadence` (:15405–15447) is a no-op for a `rounds` cadence and, for a `date` cadence, recomputes `months`, `endsAt`, `startsOn` and `endsOn` from `now`. All four are returned explicitly because the block is written with `merge:true` into a nested map: an omitted field keeps whatever the previous cadence left there. `endsOn` was the field this function once failed to return, and since the sweep asks `endsOn` first, the new season inherited the date the old one died on, was due the hour it opened, and closed again — every hour, forever, archiving an empty season each time.

`count` is read from the document inside the transaction rather than from the caller's snapshot, because `count` is what the undo button uses to decide which season to reopen (`reopenLastSeason`, `index.ts:16882–16883` : `lastId = 's' + count`). `closeSeason`'s `archived:false` latch guards the archive and nothing else, so before this transaction existed a seal-close and a sweep racing on one club both fell through and the loser rewrote the lifecycle of the season the winner had just *opened*: `startedAt` moved to now, `playedRounds` back to 0, `targetHistory` erased, `count` incremented past anything ever played. The transaction closes the race; `return advanced` then tells the caller whether it won.

`endSeasonNow` does not use any of this. It writes the equivalent block itself at `index.ts:17050–17070` — same fields, same deletes, same `rebaseCadence` — outside any transaction and with no pre-flight identity check, on the reasoning that an admin pressing a button is synchronous. It is the only close path that can run while the season has not met its target, and the only one that always stamps `endedEarly`.

Finally, `announceSeasonClosed` is called only `if (result.archived)` on both `performSeasonClose` (:15928) and `endSeasonNow` (:17071) — so a resumed close, which returns `archived: true`, will re-announce, while a redelivery, which returns `false`, will not.

[defect: partialdata-never-set-on-a-rollover-close, P2] `partialData` marks an archive whose metrics predate the counters that feed them (assists collected only from 21.06, clean sheets from 17.08). It is passed by exactly one caller — `enableClubSeasons`' "seal now" branch, `index.ts:16483`. Neither `performSeasonClose` nor `endSeasonNow` passes it, so a season 1 that is *carried on* and then rolls over is sealed with no flag at all, and the hall of fame presents its partial numbers as complete. s1 of this club is precisely that case: its archive has no `partialData` field, and its clean-sheet total of 75 covers only the portion of the season after 17.08.

6. Undoing a close

The shape of the operation

Two functions do this. `reopenLastSeason` (`functions/src/index.ts:16848–16968`) is the callable an admin presses; it owns the *lifecycle* — which season the club is on, its numbering, its finish line. `reopenSeason` (`functions/src/seasonRollover.ts:1067–1253`) is the pure-ish half that owns the *numbers* — it puts the stats back and deletes the archive. The division is stated in the doc-comment at `seasonRollover.ts:1063–1065` :

The caller is responsible for restoring the club's `seasons` block — this function owns the numbers, not the lifecycle.

That split is also the source of one of the defects below, because the two halves are not atomic with respect to each other.

The client entry point is `SeasonsSettings.tsx:525–589` (`reopenLast`), rendered as an outline button at `:971–990` , gated on `live && (seasons?.count ?? 0) > 0` . The service wrapper is `src/services/seasonService.ts:264–267` , a bare callable invocation returning `{ reopenedNo }` .

What the button is for

It is not a general time machine and the code says so twice. `index.ts:16836–16845` :

Deliberately only the LAST one, and only while it is still the club's most recent: reopening season 2 of four would leave seasons 3 and 4 sitting on numbers that were counted from a table that no longer starts where they think it does. This is a correction for "I pressed it a week early", not a time machine.

Three preconditions are enforced, in this order (`index.ts:16873–16890`):

Check	Code	Failure
Caller is a club admin	<code>requireClubAdmin(data.groupId, uid)</code>	<code>permission-denied</code>
At least one closed season	<code>seasons.count >= 1</code>	<code>failed-precondition 'no closed season'</code>
Club is quiet	<code>clubIsQuiet(groupId)</code> (no open game, nothing unsealed)	<code>failed-precondition 'openGame' \ 'unsealedGame'</code>
Archive still exists	<code>seasonSummary/{groupId}__s{count}</code>	<code>not-found 'no archive for the last season'</code>

The quiet rule exists for the same reason it exists on the close: restoring a table while an evening is mid-play folds that evening's mini-games (משחקונים) into the season being reopened, and each mini-game commits separately, so the split is per-round rather than per-evening.

Note the season the button targets is computed purely from the counter: `lastNo = seasons.count; lastId = \ s${lastNo}` (index.ts:16882-16883)`. Nothing reads `currentId` to work out what the *successor* was. The successor season is simply overwritten — it is never archived, never announced, and nothing records that it existed.

The two lists, side by side

`closeSeason` writes to seven places. `reopenSeason` reads five of them back.

#	What the close does	Where	What the reopen does	Net
1	<code>create()</code> s <code>seasonSummary/{g}__{sid}</code> — every player row, every real pair's ten counters, the totals, the nine decided titles	<code>seasonRollover.ts:584–643</code>	<code>summaryRef.delete()</code> , last	destroyed
2	<code>set()</code> s the compact <code>seasonCards/{g}__{sid}</code> for the hall of fame	<code>:752–818</code>	<code>.delete()</code>	symmetric
3	<code>set()</code> s <code>users/{uid}/seasonTitles/{g}__{sid}__{key}</code> for every winner (duo key split on __, guests skipped)	<code>:823–860</code>	<code>batch.delete()</code> on the same deterministic ids, same split, same guest skip	symmetric
4	Per player row: subtracts the 19 <code>PLAYER_SEASON_FIELDS</code> , stamps <code>seasonWoundBack</code> , deletes <code>seasonReopened</code>	<code>:878–906</code>	adds the archived row back, stamps <code>seasonReopened</code> , deletes <code>seasonWoundBack</code>	symmetric <i>for archived rows only</i>
5	Per real pair: subtracts the 10 <code>PAIR_SEASON_FIELDS</code> , stamps <code>seasonWoundBack</code> , deletes <code>seasonReopened</code> . Pairs with no archive row are zeroed . Guest pairs are deleted outright	<code>:924–1002</code>	adds back only pairs present in <code>archive.pairs</code> ; then a sweep clears <code>seasonWoundBack == seasonId</code> from every other pair row	partial
6	Club doc <code>communityStats/{g}</code> : zeroes the 7 <code>CLUB_SEASON_FIELDS</code> , deletes <code>seasonReopened</code> , re-stamps <code>chemistrySince = now</code>	<code>:1004–1022</code>	adds the archived totals back, stamps <code>seasonReopened</code> . <code>chemistrySince</code> untouched	partial
7	Lifecycle (<code>performSeasonClose</code> , <code>index.ts:15884–15925</code>): <code>currentNo+1</code> , new <code>currentId</code> , <code>startedAt=now</code> , <code>roundsAtStart=eveningsSealed</code> , <code>playedRounds=0</code> , <code>reopenedAt=0</code> , re-based cadence, <code>targetHistory=[]</code> , deletes <code>targetMovedToClose / dueBlockedSince</code> , <code>count+1</code>		<code>index.ts:16896–16966</code> restores all of it from the archive	mostly symmetric — see <code>playedRounds</code> and <code>cadence</code> below
8	<code>announceSeasonClosed</code> (<code>index.ts:15324–15372</code>) creates one <code>notifications</code> doc per participant, deep-linked to the sealed season		nothing	left dangling

#	What the close does	Where	What the reopen does	Net
9	Nothing — but every game played in the discarded successor keeps its <code>seasonId</code> stamp	<code>onGameR osterCh anged</code>	nothing	left dangling

Everything in the first column that has no entry in the third is permanent, because step 1 runs last and the archive is the only copy.

The permanent losses, enumerated

Guest pair chemistry (`reopen-never-restores-guest-pair-chemistry`). A guest id is minted per game, so `communityPairStats` accumulates one document per stranger-pairing for ever. The close deliberately keeps them out of the archive (`isReal()` at `seasonRollover.ts:31`, drop at `:436–439`) because copying them in would grow one document without bound until the club could never close a season again. Then `:943–949` deletes the guest pair documents outright. The restore loop at `:1129–1156` iterates `archive.pairs` only. So the data is gone from both sides at once. On the one club that has run seasons this is **293 of 314 pair documents, 93% of the collection** — I paged the whole collection live to confirm the counts. The module's own comment at `:1035–1041` states the rule and its consequence: *"a counter missing from this list is cleared by the close and never restored by the reopen, and the archive that held it is deleted at the end of the reopen. There is no way back from a gap here."*

Pairs past the archive cap. `MAX_ARCHIVED_PAIRS = 1200` (`:181`). Pair 1,201 onwards is zeroed by the close (`archivedPair` is `null`, so `windBackRow` writes absolute zeros — `:65–73`) and is not in the archive, so the reopen cannot restore it. For a 50-player club this is unreachable; for a large one it is silent.

`chemistrySince` (`chemistry-since-not-restored`). The close sets it to the close instant (`:1017`) so the new season's chemistry card does not date itself from the old one. The reopen never touches it. Live on the club right now: `communityStats.chemistrySince = 1789679528192 = 2026-09-17T21:12:08Z`, which is the moment `s2` was *closed*; `s2` has been running again since 10:37Z the next morning and its own `startedAt` is 21:05:41Z. The card therefore dates a running season's chemistry from six and a half minutes after it started.

Coverage denominators. `csRounds` and `asRounds` are deliberately *omitted* from the archive when the live row has no number for them (`:377–379`) — "measured across zero rounds" and "never measured" are different facts. But `windBackRow` has no matching branch: `out[f] = Math.max(0, rowNum(undefined) - 0) = 0`. So the close converts absent into `0` on exactly those rows, and the restore adds the archive's absent-as-zero back, i.e. nothing. The distinction is destroyed by the close and cannot be recovered by the undo. It affects the longest-standing members — the ones who predate the fields — and it blanks two efficiency columns for them permanently.

targetHistory of the discarded successor. The close seals the *closing* season's history into the archive (`index.ts:15838–15840`) and resets the block to `[]`. The reopen restores the archive's copy, or `[]` (`index.ts:16945–16947`). Anything the successor recorded — every finish-line move made during its life — is destroyed with no archive at all, because the successor is never archived. On this club the discarded `s3`'s `targetHistory` is gone; what the block holds today is one entry written **27 seconds after the reopen** (at `1789727874911 = 10:37:54.911Z` vs `reopenedAt: 1789727847680 = 10:37:27.680Z`) recording a rounds target moved `2 → 24`.

`seasonId` stamps on games played in the discarded season. Nothing rewrites them. Live: one finished game, `DTNscolRojYDf0ZmT0sF` at 2026-09-17T21:19:42Z, carries `seasonId: 's3'` while the club is on `s2` and no season 3 exists. Its *statistics* were folded into the live table the restore added to; its *evening* belongs to a season id that has been withdrawn.

The notification documents (`undo-leaves-dangling-season-summary-pushes`).

`createNotificationOnce` writes a durable doc per recipient; the reopen deletes none of them. Live, I fetched all seven by their deterministic ids (`seasonSummary:{uid}:group:HhzIwmjMl1i5HS0GHt3p:season-s1__b2959`): all seven exist, all seven have `read: false`, all created 2026-09-17T16:12:14Z. Pressing undo once more — which the button allows today — deletes `seasonSummary/...__s1` and leaves those seven live, unread, deep-linking into a season that no longer has an archive. And the client does not say so:

`seasonSummaryService.load` (`src/services/seasonSummaryService.ts:429-456`) tries the archive first, and when `archived.exists()` is false it falls straight through to the branch that builds the **running** season's model. The reader taps "עונה 1 הסתיימה" and is shown the current season under a heading for it.

The two latches

Neither half of this operation is idempotent — the close subtracts, the reopen adds — so each row carries a stamp naming the season it was last moved for.

```
// close, seasonRollover.ts:886-900 (player rows)
if (resuming && data.seasonWoundBack === seasonId) return;
const patch = {
  seasonWoundBack: seasonId,
  seasonReopened: admin.firestore.FieldValue.delete(),
  ...
};

// reopen, seasonRollover.ts:1101-1107 (player rows)
if (cur.seasonReopened === seasonId) return;
const patch = {
  seasonReopened: seasonId,
  seasonWoundBack: admin.firestore.FieldValue.delete(),
  ...
};
```

`seasonWoundBack` protects against a *close* being applied twice; `seasonReopened` protects against a *reopen* being applied twice. They are mutually exclusive by construction: whichever one is written deletes the other. Each is cleared by the opposite operation precisely so a *close*→*reopen*→*close*→*reopen* chain can keep working.

The `resuming &&` guard on the close side is load-bearing and recent. A `seasonWoundBack` stamp is only evidence of "already wound back" when *this invocation* wrote it — i.e. on the resume path, after the archive already exists and the wipe died halfway. On a first pass the only thing that can have left a stamp naming the

season about to be closed is a *reopen* of that same season, which handed the numbers back live while the stamp still says they were taken away. Treating it as evidence there means archiving chemistry and then skipping the wipe for it — sealing it into the write-once archive and leaving it standing live for the next season to inherit.

That is pair-wipe-skipped-on-next-close-after-reopen, and the state it feeds on is in production right now — though the defect itself is closed. All 314 pair documents on `HhzIwmjMl1i5HS0Ght3p` carry `seasonWoundBack: "s2"`, with `updatedAt: 1789679528192`, while `s2` is the running season. None carries `seasonReopened`. Two things were written to deal with this and both are deployed: the `resuming &&` qualifier above, which means the next close of `s2` winds these rows back correctly regardless of their stamps, and a stale-stamp sweep (`seasonRollover.ts:1174–1198`) that runs on *reopen* only:

```
const stamped = await db
  .collection('communityPairStats')
  .where('groupId', '==', groupId)
  .where('seasonWoundBack', '==', seasonId)
  .get();
```

The reopen that left these stamps ran at 2026-09-18T10:37:27Z; the sweep deployed at 2026-09-18T18:16:55Z (`gcloud functions describe reopenLastSeason`), seven and a half hours later, and nothing re-runs it over existing data. So the stamps stay until `s2` closes, at which point the close's own `resuming &&` guard ignores them and the 21 real pair rows are wound back properly. The 293 guest rows are not an exposure either: the close deletes them before it reaches the stamp check (`:944–948`). The residue is data that lies about itself, on a collection nothing else reads — not a pending loss.

The other asymmetry worth naming: the *player* rows only started deleting `seasonReopened` inside the close in the current HEAD (`:900`). The pair and club writes always did. Before that, a `close→reopen→play→close→reopen` cycle left every player's `seasonReopened` stamp from the *first* reopen in place, so the second reopen's per-row guard returned early for every player: club totals and pair chemistry came back, every player's season stayed at zero, and the archive was deleted on the way out. The LEDGER records this one as verified fixed in the deployed bundle.

There is no latch at all on `communityStats` for the close — it writes absolute zeros (`zeroClub, :1004–1022`) rather than subtracting, so it is naturally idempotent. The reopen does add, so it carries `seasonReopened` (`:1201–1216`). Live, `communityStats` holds `seasonReopened: "s2"` and nothing else from the pair collection does — which is itself the tell that the `s2` restore reached the club document and found no archived pairs to restore.

`playedRounds`, wrong in two directions

`seasons.playedRounds` is the mirror the club card prints as "N מתוך M מחזורים" ("N out of M game-nights") and the number the sweep closes on. It has had three implementations inside the undo path in two days.

```
// index.ts:16925-16936, as it stands today
playedRounds: await (async () => {
  const counted = await playedEveningsOfSeason(groupId, lastId, lastNo);
  if (counted >= 0) return counted;
  return typeof archive.get('completedRounds') === 'number'
    ? archNum(archive.get('completedRounds'))
    : Math.max(0, (await sealedEveningsOf(groupId)) - reopenedRoundsAtStart);
})();
```

1. `eveningsSealed - roundsAtStart`. A subtraction of two counters that drift for different reasons — `clubRecords.eveningsSealed` only exists from 26.08.2026. A season closed at 23 evenings came back holding 1.
2. `archive.completedRounds` (commit 4be7f1c, "A reopened season counts its own games, not a number it inherited"). Correct in principle — the archive is the record — and it inherits any error the archive holds. A junk season that a seeding bug opened and closed inside six and a half minutes had archived `completedRounds: 22`, the club's entire lifetime, and reopening it restored 22 onto a season that had held nothing. The club card then read "22 24 מתוך" while the statistics screen, which counts stamped games, read 0.
3. **Counted from the games** (today). `playedEveningsOfSeason` (index.ts:15599-15637) queries the club's terminal games ordered by `startsAt desc`, `limit 300` and hands them to `countSeasonEvenings` (functions/src/seasonCounters.ts:70-79), which asks `eveningInSeason` for scope and `didEveningHappen` for reality.

The third fix shipped in two halves, and the first half was itself wrong (`reopen-playedrounds-ignores-unstamped-games`). 4be7f1c filtered on `seasonId == lastId` with no notion of season 1. The stamp is only written from the day the feature shipped, so a club's pre-seasons history carries nothing — and **that history is season 1**. Nineteen of this club's twenty-two evenings are unstamped, so undoing season 1 wrote `playedRounds: 3`, which is the same defect as implementation 1 arriving from the other side. ae2c042 added the third parameter:

```
// seasonCounters.ts:53-60
export function eveningInSeason(game, seasonId, seasonNo?): boolean {
  const stamp = typeof game.seasonId === 'string' ? game.seasonId : '';
  return stamp ? stamp === seasonId : seasonNo === 1;
}
```

Run over the live games today (23 terminal games, `didEveningHappen` evaluated field by field): 3 stamped s1 happened, 19 unstamped happened, 1 stamped s3 did not (`playVerified: false`). So undoing today would write `playedRounds: 22` for s1 — matching the archive's `completedRounds: 22`, the card's 22, and the club screen's "מפגשים שנערכו". The reopen that actually ran yesterday wrote `playedRounds: 0` for s2, which is also correct: no evening was ever stamped s2.

Two caveats remain. The count is bounded at 300 documents and is *not* a query on `seasonId`, so a club with more than 300 terminal games silently under-counts an old season. And the evenings played inside the **discarded successor** are counted for nobody: their statistics were just added into the reopened season's live rows, while their games still carry the successor's stamp, so `playedEveningsOfSeason` skips them. Reopen a season after the successor has held five evenings and the club's table contains those five evenings' goals while its progress bar says they never happened.

Numbering, and the re-issued id

The reopen writes `count: closedSoFar - 1` and `currentNo: lastNo ( index.ts:16962 , :16901 )`. Since `currentNo` is always `count + 1`, the club's arithmetic stays consistent — but the *identifier* is not unique over time. The season the reopen discards had an id; nothing remembers that it was used. When the reopened season is closed again the lifecycle write allocates `s${seasonNo + 1} ( index.ts:15884 )`, which is the same id.

Live worked example. The club closed s1, opened s2, closed s2 six and a half minutes later, opened s3, then undid — so today: `count: 1, currentNo: 2, currentId: 's2'`, one finished game stamped `s3`, and `seasonSummary/...__s2` and `seasonCards/...__s2` both 404. When s2 closes again the next season will be s3 again, and that 17.09 game will be inside the new season 3 on its first day (`orphan-season-stamp-absorbed-by-the-next-season-of-the-same-id`). The same re-issue also means the write-once protections reset: `seasonSummary/{g}__s3` can be `create()` d again because the old one never existed, and `users/{uid}/seasonTitles/{g}__s3__*` will be overwritten rather than duplicated.

`undo-chains-backwards-with-no-limit`. There is no time bound and no already-undone guard. The only precondition is `count >= 1` and an archive at `s${count}`; both hold again immediately after a successful reopen, because `count` was decremented and the *previous* season's archive is intact. Two presses walk back two seasons; five walk back five, each one deleting an archive, stripping its titles off profiles, and leaving its push notifications pointing at nothing. The Hebrew confirmation (`he.ts:2234-2238`) does name the season number, and does say the archive is deleted — "אי אפשר לשחזר אותם אחר כך" ("the season summary and its archive will be deleted — they cannot be recovered afterwards") — but nothing says which press you are on, and nothing refuses a third.

The 48-hour grace

`REOPEN_GRACE_MS = 48 * 60 * 60 * 1000 ( index.ts:15455 )`. The reopen stamps `seasons.reopenedAt = Date.now() ( :16905 )` and two closing paths honour it:

```
// hourly sweep, index.ts:16178-16182
const reopenedAt = archNum(seasons.reopenedAt);
if (reopenedAt > 0 && now - reopenedAt < REOPEN_GRACE_MS) {
  console.log('[season] due but just reopened - holding', doc.id);
  continue;
}
// close-on-seal, index.ts:15977-15979 - same two lines
```

The reason is that a season is usually reopened *because it met its target*, so it is due the instant it comes back; without the grace the sweep would re-close it within the hour and the button would be inert. The grace is a window for the admin to move the finish line. It is cleared by the next close (`reopenedAt: 0` , `:15906`) and by enabling seasons on a club that had them off (`:16502`) — but *not* by `disableClubSeasons` , which writes only `{ seasons: { enabled: false } }` (`:16622`). A club that reopens, disables and re-enables within 48 hours hands its brand-new season the tail of somebody else's grace (`reopenedAt-survives-disable-enable`).

The grace protects against re-closing. It does **not** protect the *cadence*.

`reopen-extends-a-date-season-by-a-full-length` . `index.ts:16957-16961` re-bases the restored cadence through `rebaseCadence` (`:15405-15446`) — the function written for opening the *next* season:

```
cadence: rebaseCadence(
  (archive.get('originalTarget') ?? seasons?.cadence) as never,
  archNum(archive.get('startsAt')),
  Date.now(),
),
```

For `type: 'rounds'` this is a pass-through and is right. For `type: 'date'` it computes `months` , then sets `endsAt = addMonthsClampedServer(now, months)` , `startsOn = todayIn(now)` , `endsOn = seasonEndDate(startsOn, months)` . A six-month season closed on its end date and undone the next morning is given six more months *from today* — twelve months in total — and its `startsOn` is moved to today while `startedAt` is restored to the original start from the archive. The season then reports a start six months ago and a calendar window that begins today. The comment at `:16955-16956` says the re-base exists "so it is not immediately due again on a date cadence", which is true and is also a much larger intervention than the 48-hour grace it duplicates.

The window where the club is double-counted

`club-seasons-block-written-only-after-the-restore-succeeds` . The order is: run `reopenSeason` to completion — including `summaryRef.delete()` as its final act (`seasonRollover.ts:1252`) — and only then `ref.set({ seasons: {...} })` (`index.ts:16896`). There is no transaction spanning the two, and no compensating write.

If the invocation dies in between (the callable's ceiling is `timeoutSeconds: 300` , and the work is one transaction per player plus one per pair plus a batched title delete plus a collection scan), the club is left in a state that cannot be repaired by the same button:

- every player row and the club totals hold the reopened season's numbers **added on top of** the successor's, and are stamped `seasonReopened: s{n}` ;
- `seasonSummary/...__s{n}` and `seasonCards/...__s{n}` are gone, so nothing can say what was added;
- the club block still says `currentId: s{n+1}` , `count: n` ;
- a retry throws `not-found 'no archive for the last season'` at `index.ts:16889` , because the archive it needs to read is the one just deleted;
- even if the archive were restored by hand, every row's `seasonReopened` guard would return early.

The failure is silent, permanent, and doubles the club's whole table. Reversing the two — lifecycle first, archive delete last — would make the operation resumable, which is the property the close's own `create()` -then-wipe ordering was built around (`seasonRollover.ts:1-16`).

One more thing the re-close does not do

`reclose-after-undo-notifies-nobody` . `announceSeasonClosed` routes through `createNotificationOnce` , whose doc id is `dedupeKey + '__b' + floor(now / cooldown)` and whose cooldown for `seasonSummary` is **seven days** (`functions/src/notificationDedup.ts:95`). The write is a `create()` , and `AlreadyExists` is swallowed. The undo deletes the archive but not the notification documents, so closing the same season again inside the same seven-day bucket collides on every id and reaches nobody. The toast still says "הסתיימה ונשמרה בארכיון N עונה" ("season N has ended and was saved to the archive"), which is true, and the admin has no way to learn that the one push the whole feature builds towards was suppressed.

7. Where the numbers come from, and who can change them after the fact

The sections before this one describe what a season *does* with its numbers. This one describes where those numbers come from, who is allowed to change them after the fact, and what happens to a season when the evening underneath it is deleted, confirmed late, or corrected. The scale figures it uses, read live: **195** groups , **93** communityPlayerStats rows across **12** clubs, **717** communityPairStats , **95** games, **1** seasonSummary , **1** seasonCards .

Throughout this section, "deployed" means the server — which is HEAD — and "shipped" means the phone, which is eleven commits behind it. The front matter sets out the evidence for both.

The two functions that write every counter a season archives

closeSeason does not compute anything. It copies communityPlayerStats , communityStats and communityPairStats into a document and subtracts them back out. Everything a season *is* was written earlier by two functions, and no other section names either of them.

commitRoundStats — functions/src/index.ts:13800 , a callable, one invocation per משחקון (mini-game) committed from the advanced live screen. It is the sole writer of fifteen of the nineteen PLAYER_SEASON_FIELDS and of all seven CLUB_SEASON_FIELDS :

target	fields	line
communityPlayerStats/{g}__{uid}	goals , assists , ownGoals	14141–
	rounds , cleanSheets , csRounds , asRounds	14337, 14356-14357
	wins / losses / ties	14440–
	penTaken/penScored/penMissed/penFaced/penSaved/penConceded	14200–
communityStats/{g}	rounds , goals , guestGoals , ownGoals , tiedRounds , shootoutRounds , scorelessRounds	14162-14189
communityPairStats/{g}__{a}__{b}	assists only	14474

CLUB_SEASON_FIELDS is that list at 14166-14186, field for field, in order. The archive's totals map is a transcript of one sb.bump call.

Note the coverage denominators ride the *same* write as the metric they divide, at zero extra Firestore operations — which is why they exist at all, and why their absence on an old row is information rather than an omission:

```
// functions/src/index.ts:14337-14358 — one bump per on-field player per mini-game
sb.bump(cpsRef(uid), { groupId, userId: uid, updatedAt: now }, {
  rounds: 1,
  ...(clean ? { cleanSheets: 1 } : {}),
  csRounds: 1,
  asRounds: 1,
});
```

`rollUpClubPairs` — `functions/src/index.ts:4695-4788`, and this is the one that surprises. `commitRoundStats` writes exactly **one** of the ten `PAIR_SEASON_FIELDS`. The other nine — `sameTeam`, `winsTogether`, `lossesTogether`, `cleanSheetsTogether`, `against`, `winsA`, `winsB`, `assistsAToB`, `assistsBToA` — are written once **per evening**, after the evening is credited, by a separate rollup that re-reads the mini-games from a subcollection:

```
// index.ts:6466-6496, inside the creditedNow block of onGameRosterChanged
const rhSnap2 = await db.collection('games').doc(gameId).collection('roundHistory').get();
const rounds2: ChemistryRound[] = rhSnap2.docs.map(...);
await rollUpClubPairs({ gameId, groupId: gid, at: after.startsAt, rounds: rounds2 });
```

`rollUpClubPairs` is latched by `communityPairRollups/{groupId}__{gameId}` (:4710-4716), chunks its writes at 450 ops, and computes the pair totals with `pairsFromRounds` (`functions/src/clubChemistry.ts:95-146`) — collapsing the whole evening in memory first, because writing pairs per mini-game adds n^2 operations to a batch already near Firestore's 500 ceiling, *and the idempotency latch is inside that batch*, so an overflow would lose the round's statistics permanently. It also stamps `communityStats.chemistrySince` the first time it runs (:4772-4777).

Two consequences the other sections observe as symptoms without naming the cause.

Guests. `pairsFromRounds` treats a guest as a full participant — its comment says so: *"they were on the pitch, the pass was real... they simply have no account, which matters for titles, not for what happened."* `closeSeason` then drops every guest pair (`isReal`, `seasonRollover.ts:31`, :436). So the collection that grows without bound and the collection the archive keeps are governed by two deliberate and *opposite* decisions, taken in two files, neither of which cites the other. 293 of the live club's 314 pair documents are the result.

The assist gap. §10 reports that the archive's directional pair assists sum to 7 against `totals.assists: 13`, and calls the cause unexplained. It is the two writers' two windows. I read every `roundHistory` document of all 23 surviving games of `HhzIwmjMl1i5HSOGht3p` and counted the goals that carry both a scorer and a different assister and are not own goals:

```

roundHistory documents across the 23 games : 31
committedRounds latch documents           : 35
communityStats.rounds sealed into s1      : 37
assisted goals recoverable from roundHistory: 7    (0 of them involve a guest)
archive pairs, assistsAToB + assistsBToA   : 7    ← exact match
archive pairs, legacy undirected `assists` : 8
archive players[].assists / totals.assists: 13 / 13

```

Four mini-games were committed with no `roundHistory` document at all — three on `WhqMQzLznMgP` (28.06) and one on `5EQKVHge5mtP` (29.06), the club's two oldest evenings, which predate the subcollection — and two more belong to games that have since been deleted (below). The pair rollup can only see what `roundHistory` holds, so those mini-games contributed **no chemistry at all**: no `sameTeam`, no `against`, no directional assist. The legacy `assists` is 8 rather than 7 because `commitRoundStats` writes it at commit time from the payload in hand, so it saw one mini-game the rollup did not. Three numbers, three windows, one Hebrew word.

Nothing in the repository reconciles `communityStats.rounds` (37) against the `committedRounds` latches (35) or `roundHistory` (31). `totals.rounds: 37` is the number `he.seasonHistoryLine` prints as "37 משחקונים" on a permanent card. This is open question 8 in §13: three counts of one quantity, and no owner.

`games` is credited without asking which season it is

The nineteenth `PLAYER_SEASON_FIELD`, and the only one every club records, is written somewhere else again — in `onGameRosterChanged`, on the `didEveningHappen` `false→true` transition (`index.ts:5868–5919`):

```

const wasHappened = didEveningHappen(before);
const isHappened  = didEveningHappen(after);
if (!wasHappened && isHappened && after.groupId && after.players?.length) {
  for (const uid of after.players) {
    if (arrivals[uid] === 'no_show') continue;
    batch.set(db.collection('communityPlayerStats').doc(`${gid}__${uid}`),
      { groupId: gid, userId: uid, games: increment(1), updatedAt: Date.now() }, { merge: true })
  }
  batch.create(db.doc(`games/${gameId}/finishCredited/once`), { at: Date.now() });
}

```

Three things follow, and the third is not written down anywhere.

1. **The `finishCredited/once` marker is the audit trail for §9's 22-vs-19.** I probed it on all 23 games: **19 exist**. The four without one are `2SMrlCGH` (06.07), `MMtE8J6H` (16.07), `FCa5UtSd` (04.08) — the three happened evenings that never advanced past `liveMatch.phase: 'organizing'` — and `DTNscoLR` (17.09), which is `notHappened`. 22 happened evenings minus 3 uncredited = the 19 in the archive, exactly.
2. No-shows are excluded here, and the client scan excludes them too (`gameService.ts:1206–1210`), so the two do **not** diverge on that axis. The whole of the 3-evening gap is the missing marker.

3. **There is no season check.** The `mine test ( index.ts:5092–5106 )` gates `seasons.playedRounds` and `roundSummaries.seasonEvenings`; it does not gate this. An evening whose stamp names an archived season, or an orphan season, still adds `games: +1` to the *running* season's live rows. `games/DTNscolRojYDf0ZmT0sF` is stamped `s3` on a club that has only ever had `s1` and `s2`, is `playVerified: false`, and has no `finishCredited` marker — so it is one admin tap away from crediting seven attendances into season 2's live table while season 2's own progress counter (`playedRounds`, currently `0`) refuses to move and no season's archive will ever contain the night.

`setEveningPlayed` — how an evening joins a season days later

`index.ts:13704–13790`. Admin-only (`adminIds`, not the creator), and the read and write are in one transaction because two admins can be looking at the same screen and a "no" is unrecoverable.

```
const correctingAMistap = state === 'notHappened' && cur.playVerified === false && played;
if (state !== 'unverified' && !correctingAMistap) return { changed: false, state };
tx.update(ref, { playVerified: played, playVerifiedBy: uid, playVerifiedAt: Date.now(), ... });
```

The transition is a plain field write with **no status change**, which is precisely why the credit block above keys on `didEveningHappen` rather than on status. It is also why the season *stamp* cannot be written by it: the stamp's gate is `before.status !== after.status ( index.ts:5802–5810 )`, so a late-confirmed evening that was never stamped stays unstamped for ever and is filed under season 1 by every reader. On a club past season 1 that is an evening credited to the live table and attributed to a sealed archive. ROUND4 §15 covers what the *seal* then computes from today's totals; the seasons half of it is untested and unnamed.

The fourth copy of the season-membership rule

`inSeason` (client) and `eveningInSeason` (server) are documented as the rule. There is a third copy, and it is the only place in the product where `game.seasonId` *refuses* an action:

```
// functions/src/index.ts:14924–14937, loadRetroGameContext – shared by addRetroGoal + removeRetroGoal
const gameSeason = typeof game.seasonId === 'string' ? game.seasonId : '';
const seasons = grp.seasons as { enabled?: boolean; currentId?: string; currentNo?: number } | undefined;
if (seasons?.enabled) {
  const belongsTo = gameSeason || (seasons.currentNo === 1 ? seasons.currentId ?? '' : 's1');
  if (belongsTo !== seasons.currentId) {
    throw new HttpsError('failed-precondition',
      'closedSeasonGame: this evening belongs to a season that has already closed');
  }
}
```

Retro goals write straight into `communityPlayerStats.goals/assists`, `communityStats.goals` and `users.stats ( index.ts:14985–15015 )` with no round attached, so correcting a June goal after a close would credit it to the running season while the sealed archive stayed wrong — and `removeRetroGoal` would decrement a counter the close has already set to zero, since Firestore's `increment` goes negative happily. The guard is right to exist. Three things about it:

- It spells the unstamped→season-1 rule a **fourth** way, by hard-coding the literal 's1' rather than comparing season numbers. It happens to agree with the other three because ids are always s\${no} , but nothing pins that.
- It is inert when seasons.enabled is false, so a club that switched seasons off can still retro-credit goals into a table a close zeroed.
- Its admin gate is game.createdBy === uid || adminIds.includes(uid) (:14899), which is **wider** than every seasons callable's requireClubAdmin (index.ts:15710–15721, adminIds only). A non-admin who created the game can move the club's season counters.
- closedSeasonGame is mapped by seasonRefusalText (seasonService.ts:119) to he.seasonBlockedClosedGame , but it is thrown from a **match** screen, not a seasons screen — RetroGoalsSheet.tsx:47 has its own copy of the substring test.

Production: games/fnBXPo76y0lm (14.07.2026) carries one retroGoals document. That evening is unstamped, so belongsTo is 's1' and currentId is 's2' — **an admin can no longer remove that retro goal**, and the counter it added was archived into s1 and then wound back out of the live table. games/DTNscolRoj , stamped s3 , is refused for ever by construction.

This is also the mechanism behind the title arithmetic in §8. addRetroGoal writes goals and assists onto communityPlayerStats **without** writing rounds , so a club that never opens the advanced screen can still accumulate season goals and assists, and can therefore award topScorer and topAssister on top of mostLoyal . That is why the Hebrew comment's figure of three reachable titles for a timer-only club is right — a reading of commitRoundStats alone gives one, and is wrong.

Deleting an evening

onGameRosterChanged's deletion branch (index.ts:5355–5428) writes a gameDeletions/{gameId} audit row and fans out a cancellation push. It touches **no** counter: not clubRecords.eveningsSealed , not seasons.playedRounds , not communityPlayerStats , not communityStats , not communityPairStats . §3 states this in the abstract. It has happened four times on the one club that runs seasons:

doc	deletedAt (UTC)	rosterCount	source
QKMctm2oa0BpD8z5bNMj	2026-08-04 10:58:29	7	manual
HVJ1QfFk3SWN4TyAGmjF	2026-09-02 06:35:26	6	manual
oi6Iz1F308lkbHrv56vy	2026-09-15 06:53:00	0	manual
Iyj4md7E46cbeYL8v61T	2026-09-17 15:13:42	6	manual

All four by the same admin (YIZlKWbvvjae3oqgIoAMr9nzQEi1 , Eliran Tzabari). The last one was deleted **59 minutes before season 1 closed** at 16:12:06Z. Whatever that evening contributed to communityStats.rounds , communityStats.goals and every player's row was still there at 16:12 and was sealed into a write-once archive; the client's games scan lost it the moment the document went. This is the mechanism behind two numbers §3 calls coincidental: the counters and the scan on this club drift *both* ways at once — the counters are blind to twelve evenings before 26.08, and the scan is blind to four deletions.

`promoteOrphanToGroup` is the one other path that destroys seasons inputs. Its artefact purge (`index.ts:9322–9358`) deletes every `communityPairStats` row, every `eveningStandings` row, every `roundSummaries` and `communityPairRollups` marker, and `clubRecords/{groupId}` — the document holding `eveningsSealed`, which is the zero mark every `roundsAtStart` is measured from. It does not touch `seasonSummary`, `seasonCards`, `users/*/seasonTitles` or `groups.seasons`. A personal group cannot hold a seasons block today (`firestore.rules:334–341`), so the combination is unreachable; it is one rule change away from a club whose archives outlive the counter they were measured against.

The season picker

`src/utils/seasonChoices.ts` — 52 lines, pure, and the only thing that decides which seasons a person is offered on `SeasonSummaryScreen` and whether `SeasonsCard` draws its history button.

```
const MAX_SEASON_CHOICES = 200;
export function seasonChoices(seasons: GroupSeasons): SeasonChoice[] {
  const out = [{ no: seasons.currentNo, id: seasons.currentId, closed: false }];
  const closed = Math.max(0, Math.min(MAX_SEASON_CHOICES, seasons.count ?? 0));
  for (let no = closed; no >= 1; no -= 1) {
    if (no === seasons.currentNo) continue; // disable/re-enable keeps numbering
    out.push({ no, id: `s${no}`, closed: true });
  }
  return out;
}
```

It derives ids from `count` alone and never reads an archive, so **the picker asserts that `seasonSummary/{g}__s{n}` exists for every $n \leq \text{count}$** . That assertion is exactly what a reopen breaks in the other direction: the reopen decrements `count` (`index.ts:16962`), so the discarded successor disappears from the list with no trace, which is correct — but nothing removes a chip for an archive that was deleted by hand or never written. `CommunityStatsScreen` handles the miss by bailing the whole scope back to `{k: 'current'}` (`:316`); `SeasonSummaryScreen` handles it by silently rendering the **running** season under the closed season's heading (`seasonSummaryService.ts:429–456`). Two screens, two different answers to one missing document.

`lastClosedSeason` (`:48–52`) is `seasonChoices(...).find(c => c.closed)`, with a first-line guard that returns `null` for a club that has neither `enabled` nor a `count`. It is what gates the "שלי N סיכום עונה" button.

Observability: eight events, and the three that do not exist

event	where	payload
<code>SeasonsEnabled</code>	<code>SeasonsSettings.tsx:502</code>	<code>groupId</code> , <code>cadence</code> , <code>target</code> , <code>sealedHistory</code>
<code>SeasonsDisabled</code>	<code>:479</code>	<code>groupId</code>
<code>SeasonTargetChanged</code>	<code>:516</code>	<code>groupId</code> , <code>cadence</code> , <code>target</code>

event	where	payload
SeasonReopened	:557	groupId, seasonNo
SeasonEndedEarly	:612	groupId, seasonNo
SeasonHistoryViewed	SeasonHistoryScreen.tsx:319	groupId, seasons
SeasonSummaryViewed	SeasonSummaryScreen.tsx:308	—
SeasonSummaryShared	:349	—

Every one is client-side and every one is an **admin action or a screen open**. There is no event for a season closing by the hourly sweep, none for a close-on-seal, none for the summary push. The two mechanisms that close seasons without a human — the two that also destroy data — emit nothing to analytics, log to `console.log`, and (per §12) have never written a row to the `errors` collection. The only durable record that an automatic close happened is the archive it creates.

The mock layer, and why QA cannot see the common club

`USE MOCK DATA` short-circuits all three seasons services: `seasonService` returns success without calling anything (:234–266), `seasonHistoryService` returns `mockHistory()` / `mockTable()` (:151–233), `seasonSummaryService` returns `mockSeasonSummary()` (:549–).

Every mock player in every one of the three builders has `rounds > 0` — 22, 24, 20, 12 on the closed season; 41, 44, 38, 30 on the running one — and every mock pair has non-zero `sameTeam` and directional assists. **The timer-only club is unreachable in mock mode**. That is the club the product notes call common; it is the club for which `rank-denominator-of-excludes-attendees`, `share-card-is-all-zeros-for-a-timer-only-club`, `seasonsummary-participant-rounds-only` and `season-push-body-promises-three-things-a-timer-club-has-none-of` were all written. Mock QA cannot reproduce any of them.

The mock is also the **only** place two archive fields are ever exercised: `mockHistory()` sets `partialData: true` (`seasonHistoryService.ts:161`), and no production archive carries the field at all, so the "נתונים חלקיים" chip has only ever rendered against a fixture. `mockTable()` deliberately includes one uid that is *not* in the mock roster (`'u_left_the_club'`, :224) to exercise the frozen-name path — a good instinct that the real fixture (`tests/fixtures/realClub.ts`) does not copy.

Three small things with no owner

`src/utils/seasonTitleIcon.ts` (51 lines) holds `SEASON_TITLE_ICON` and `SEASON_TITLE_TINT`, total `Record<SeasonTitleKey, ...>` maps so a tenth title is a compile error. The tints are declared usable **only on a disc, never as text colour, where they fail contrast** — a constraint nothing enforces. It is the fourth total record keyed on `SeasonTitleKey`, after `SCALE` (`seasonMedalTier.ts:57`), `TITLE` (`he.ts:34`) and `SEASON_TITLE_KEYS` itself.

`parseSeasonTable` **does not read** `completedRounds`. `FinishedSeasonTable` (`seasonArchive.ts:35–48`) has seven totals, a players array, a duo and a names map — and no season length, no `no`, no `awards`, no `endsAt`. So the archive has two client readers with two disjoint field sets: `seasonHistoryService.table` → `parseSeasonTable` for the stats table, and `seasonHistoryService.list` → `fromCard` over `seasonCards` for everything else. A screen that wants the length of a closed season must read the **card**, never the archive it is looking at.

`he.seasonRangeUntil` ("מתחילת המועדון עד X" — "from the club's beginning until X") is now reachable only when `startsAt === 0`, which only the `sealNow` activation branch produces (`index.ts:16478`). No production season has it. ROUND4 §19 is closed; the string is now dead code for every club that carried its history rather than sealing it.

8. The nine titles

Where the code lives

The whole award decision is one pure function in one file, duplicated byte-for-byte on the server:

file	role	lines
<code>src/utils/seasonAwards.ts</code>	the canonical copy, bundled into the app	1–247
<code>functions/src/seasonAwards.ts</code>	the deployed copy, a 15-line header then an exact transcript of the client file	1–15 header, 16–262 copy
<code>tests/logic/seasonAwardsMirror.test.ts</code>	fails the build if the two ever drift	—
<code>functions/src/seasonRollover.ts</code>	the only caller: builds the input rows, calls it, seals the result	503–560
<code>src/utils/seasonMedalTier.ts</code>	grades a won title into bronze/silver/gold/platinum	1–177

Because of the 15-line header, every line number in the server mirror is the client line number +15. All line numbers below refer to the client file unless stated.

Nothing on the client ever calls `computeSeasonAwards`. The titles are decided exactly once, inside the close, and written into three places: `seasonSummary/{groupId}__{seasonId}.awards` (the archive), `seasonCards/{groupId}__{seasonId}.winners` (the denormalised card the hall of fame reads, `seasonRollover.ts:715–732`), and one document per winner at `users/{uid}/seasonTitles/{groupId}__{seasonId}__{titleKey}` (`seasonRollover.ts:831–859`). The client file is compiled and shipped, but on the app side it is used only for its `SeasonTitleKey` type and `SEASON_TITLE_KEYS` ordering.

The input rows, and what actually writes each field

`computeSeasonAwards` takes three arguments and reads nothing else:

```
// src/utils/seasonAwards.ts:171
export function computeSeasonAwards(
  players: readonly SeasonPlayerLine[],
  pairs: readonly SeasonPairLine[],
  completedRounds: number,
): SeasonAwards
```

`SeasonPlayerLine` (lines 26–52) is assembled at `seasonRollover.ts:503–522` from the live `communityPlayerStats/{groupId}__{uid}` rollup rows, which are read at `seasonRollover.ts:310–313` before the close wipes them. The provenance of each field is the single most important fact in this section,

because it decides which titles a club can win at all:

field	unit	written by	present for a timer-only club?
games	evenings (מחזורים)	onGameRosterChanged , functions/src/index.ts:5901 , +1 per attendee per sealed evening, no-shows excluded	yes
rounds	mini-games (משחקונים)	commitRoundStats , index.ts:14330–14338	no
goals , assists , wins , cleanSheets	mini-games	commitRoundStats	no
penTaken/penScored/penFaced/penSaved	kicks	commitRoundStats §1c	no
mvpAvg	evening score 6–10	derived at seasonRollover.ts:516–519 from eveningScoreSum / eveningScoreCount , both incremented at index.ts:6378–6379	yes — but always exactly 6.0 (below)

commitRoundStats is the advanced-mode mini-game commit. A club that runs the plain live timer never calls it, so for the common club **six of the nine titles are decided on fields that are permanently zero**: topWinner , cleanSheetKing , penaltyKing , penaltyKeeper , deadlyDuo , and — because the evening score returns its 6.0 sentinel when a player has no mini-games, and MVP_SCALE_FLOOR is a strict > 6 — mvp . Three remain reachable. mostLoyal reads games , which every club records. topScorer and topAssister read goals and assists , which commitRoundStats is not the only writer of: addRetroGoal (functions/src/index.ts:14985–15015) increments both on communityPlayerStats with no rounds attached, so a timer-only club whose admin corrects a missed goal after the match can and does accumulate season goals and assists. The Hebrew settings copy at src/i18n/he.ts:2103–2107 says such a club "can never award more than three" (לא יכול לחלק יותר משלושה), and three is right. §7 sets out the retro-goal path and its own season guard.

SeasonPairLine (lines 55–62) is built at seasonRollover.ts:525–537 from communityPairStats :

```
score: p.assistsAToB + p.assistsBToA,    // directional assists, both ways
together: p.sameTeam,                    // mini-games on the same side
```

leaders() — the single ranking primitive

Every one of the nine goes through the same eight-line function (lines 144–161):

```
function leaders<T>(rows, value, id, floor = 0): SeasonAward | null {
  let best = -Infinity;
  for (const r of rows) { const v = value(r); if (v > best) best = v; }
  if (!Number.isFinite(best) || best <= floor) return null;
  const winners = rows.filter((r) => value(r) === best).map(id);
  return winners.length ? { winners, value: best } : null;
}
```

Three properties follow, and they are the whole ranking policy:

1. **Ties are shared, always.** Every row on the maximum is returned. There is no uid tie-break anywhere in this file — deliberately, per the header comment at lines 18–20. This is the opposite of the club-screen leaders (see below), and it is why a single sealed season in production carries a title with seven holders.
2. **The floor is strict** (`best <= floor` $\rightarrow$ `null`). With the default `floor = 0`, "everyone on zero" yields *no title*, not a winner on zero. `null` is written into the archive and rendered as 'לא זוכה' ("not awarded", `he.ts:2496`) in a fixed nine-slot cabinet.
3. **Equality is exact float equality.** The comment at lines 157–158 states this is intentional so 8.3746 and 8.3751 are two numbers. For integer titles this is harmless. For `mvp`, which is a quotient of floats, it means a shared MVP is essentially unreachable for real data ($21.6/3 = 7.199999999999999$), while the *degenerate* sentinel case ties perfectly — see below.

The nine, one by one

`SEASON_TITLE_KEYS` (lines 83–93) fixes both the type union and the render order of the medal cabinet; `SeasonHistoryScreen.tsx:143–146` chunks it three-per-shelf, so a title nobody won is drawn as an empty socket rather than closing the gap.

key	Hebrew (<code>he.ts:34–43</code>)	measures	field read	unit	floor	eligible set
<code>topScorer</code>	כתר השערים	goals scored	<code>p.goals</code>	mini-game goals	<code>> 0</code>	eligible players
<code>topAssister</code>	כתר הבישולים	assists	<code>p.assists</code>	mini-game assists	<code>> 0</code>	eligible players
<code>mvp</code>	כתר העונה	mean evening score	<code>p.mvpAvg</code>	6–10 score	<code>> 6</code> (<code>MVP_SCALE_FLOOR</code> , line 141)	eligible players
<code>topWinner</code>	כתר הניצחונות	mini-games won	<code>p.wins</code>	mini-games	<code>> 0</code>	eligible players
<code>mostLOYal</code>	כתר ההתמדה	evenings attended	<code>p.games</code>	evenings	<code>> 0</code>	eligible players

key	Hebrew (he.ts:34–43)	measures	field read	unit	floor	eligible set
cleanSheetKing	כתר השער הנקי	mini-games whose side conceded nothing	p.cleanSheets	mini-games	> 0	eligible players
penaltyKing	כתר הפנדלים	conversion rate	penScored / penTaken	ratio 0–1	> 0	eligible and penTaken >= minAttempts
penaltyKeeper	כתר העצירות	save rate	penSaved / penFaced	ratio 0–1	> 0	eligible and penFaced >= minAttempts
deadlyDuo	הצמד הקטלני	assists exchanged between two players	assistsAToB + assistsBToA	assists	> 2 (MIN_DUO_ASSISTS – 1 , line 244)	pairs where both halves are eligible

Every one of them shares ties; there is no per-title tie-break to document, because there are none.

Two units are worth flagging for the reader, because the product's own vocabulary collides here. `mostLoyal` counts **evenings** and is rendered 'מחזוריים' (he.ts:2443–2444). `cleanSheetKing` , `topWinner` , `topScorer` , `topAssister` count **mini-games** or events inside them. The same season card therefore prints "19 מחזוריים" beside "13 שערין נקיים" where the 19 counts evenings and the 13 counts mini-games out of 37 — two denominators, neither shown.

`deadlyDuo` is the only title whose id is not a uid: `leaders` is given ``${p.a}__${p.b}`` (line 243), and the title-write loop at `seasonRollover.ts:837` splits that key back apart to place one document on each of the two profiles, skipping any half whose id starts with `guest:` .

Defect — `deadlyduo-together-is-collected-and-never-read` . `SeasonPairLine.together` (line 61, "Rounds the two were on the same side") is populated at `seasonRollover.ts:536` from `p.sameTeam` and is never read by `computeSeasonAwards` — `grep -n together src/utils/seasonAwards.ts` returns exactly one hit, the field declaration. The partnership-volume gate the field exists for was never written. A pair that played together twice and exchanged three assists outranks a pair that played together fifteen times and exchanged three, and the code has the number to distinguish them sitting unused in the same object.

The eligibility gate

```
// lines 102–117
export function eligibilityThreshold(completedRounds: number): number {
  return Math.ceil(Math.max(0, completedRounds) / 2);
}
export function isEligible(line: Pick<SeasonPlayerLine, 'games'>, completedRounds: number): boolean {
  return line.games >= eligibilityThreshold(completedRounds);
}
```

Applied once at line 190 (`const eligible = players.filter(...)`) and inherited by all nine titles. Pairs inherit it transitively at lines 195–198: both halves must clear it on their own, so a regular and a one-night guest cannot take the duo between them.

A zero-length season short-circuits before any of this (lines 184–189) and returns nine `null`s, because `eligibilityThreshold(0) === 0` would otherwise open every gate.

The user-facing promise is at `he.ts:2416` :

'...לא חולקו תארים בעונה הזאת. תואר ניתן רק למי שהגיע לפחות לחצי מערבי המשחק של העונה' "No titles were awarded this season. A title goes only to whoever attended at least half of the season's game evenings."

Now the subtlety that makes that sentence false. The `completedRounds` argument is *not* the season's length. The caller computes it at `seasonRollover.ts:556–559` :

```
const seasonEvenings = awardsDenominatorOf(
  awardLines.map((l) => l.games),
  args.completedRounds,
);
const awards = computeSeasonAwards(awardLines, awardPairs, seasonEvenings);
```

and `awardsDenominatorOf` (`seasonRollover.ts:133–139`) is:

```
export function awardsDenominatorOf(attendances: readonly number[], fallback: number): number {
  if (attendances.length === 0) return Math.max(0, rowNum(fallback));
  return Math.max(0, ...attendances.map((g) => rowNum(g)));
}
```

The denominator is **the maximum attendance of any single player**, and the true season length (`args.completedRounds` , ultimately `clubRecords.eveningsSealed` minus the season's starting offset, via `completedRoundsOf` at `index.ts:15695–15708`) is used only as a fallback when the club has no player rows at all. The stated rationale, written out at `seasonRollover.ts:114–121` , is a units/era argument: per-player `games` has been counted since 22.06 while the sealed-evening counter was only born on 25.08, so comparing a numerator from one era against a denominator from the other admitted far too many players. `max(games)` is at least in the same unit and the same era as the numerators it is compared against.

The consequence is algebraic and unconditional. Let $D = \max_i(\text{games}_i)$ and let p^* be a player attaining it.

- `isEligible(p*, D)` is `games_{p*} >= ceil(D/2)` , i.e. $D \geq \text{ceil}(D/2)$, which is true for every $D \geq 0$. **The top attendee can never fail the gate.**
- `mostLoyal = leaders(eligible, p => p.games)` returns `best = max` over the eligible set. Since p^* is always in that set, `best = D` . `awards.mostLoyal.value === D` **identically, for every attendance distribution.**

So the gate's denominator is, by construction, the value of one of the titles it gates — and it is a *different number* from the season length that the same close writes to `seasonSummary.completedRounds` (`seasonRollover.ts:625`) and `seasonCards.completedRounds` (:765).

On the real archive. `seasonSummary/HhzIwmjMl1i5HS0Ght3p__s1` , read live:

```
players[*].games = { 1IdtNEjb: 19, alsobLSA: 18, K5rSGB4J: 18,
                    B5KpY04I: 18, CEkRfDs2: 18, YIZlKWbV: 18, JoLFRxFr: 17 }
completedRounds  = 22
awards.mostLoyal = { winners: ['1IdtNEjb...'], value: 19 }
```

`D = 19 ; mostLoyal.value = 19 ;` the season was 22 evenings long. The gate that the copy calls "half the season" was `games >= ceil(19/2) = 10` , i.e. **10 of 22 = 45%**. All seven members cleared it either way here, so on this club the gate's membership did not change — but a title did, which is the point below.

Defect — eligibility-denominator-is-max-attendance . Two numbers describing "how long was this season" live in one archive document (19 and 22) and neither is labelled as the other's alternative; `awardsDenominator` is no longer published anywhere (see below), so a reader of the sealed archive cannot tell which denominator the titles were decided on without re-deriving `max(games)` by hand.

Defect — loyalty-crown-is-structurally-platinum (fixed at HEAD, and worth the reader's attention as a worked example of the identity). For about a day, `seasonRollover.ts` wrote `awardsDenominator: seasonEvenings` onto the season card and `SeasonHistoryScreen.tsx` graded the loyalty medal as `medalTier('mostLoyal', w.value, season.awardsDenominator ?? season.completedRounds)` . Because `w.value === awardsDenominator` identically, the ratio was **1.0** for every season of every club, and the attendance scale's platinum stop (`>= 1`) fired unconditionally: silver and gold became unreachable code. Both the field and the read are gone at HEAD — `SeasonHistoryScreen.tsx:172` is now `medalTier(key, w.value, season.completedRounds)` and `seasonRollover.ts:786–800` carries a long comment explaining why the field must not come back. The production `seasonCards/HhzIwmjMl1i5HS0Ght3p__s1` predates the field and does not carry it. **Nothing recomputes a sealed season**, so this is history rather than live damage — but the *gate* still uses `max(games)` , which is the same identity in the place it was not removed from.

The volume gates

```
// lines 126–128
export function minPenaltyAttempts(completedRounds: number): number {
  return Math.min(5, Math.max(2, Math.ceil(Math.max(0, completedRounds) / 10)));
}
```

Two attempts minimum, one more per ten "rounds", capped at five. Because the argument is `max(games)` rather than the season length, this gate moves with the club's best attendee. **On production s1 this flipped a real title.**

denominator	minPenaltyAttempts	qualifying kickers	rates	penaltyKing
<code>max(games) = 19</code> (what ran)	2	alsobLSA 2 taken, K5rSGB4J 3 taken	2/2 = 1.000 , 2/3 = 0.667	הלן צברי , value 1.0
<code>completedRounds = 22</code>	3	K5rSGB4J only	2/3 = 0.667	Nofar Tzabari, value 0.667

The sealed archive holds `penaltyKing: { winners: ['alsobLSA...'], value: 1 }` — reproducing the stored result requires 19, not 22, which is independent confirmation that the denominator in use is the maximum attendance. (`penaltyKeeper` is unaffected: at both 2 and 3 the qualifying keepers are the same three, and alsobLSA's 3 saves from 3 faced wins either way.)

Defect — `penalty-titles-crowned-on-two-kicks`. A season crown at a perfect 100% off **two kicks** is what the archive actually contains, and `medalTier('penaltyKing', 1.0, ...)` grades it platinum (`RATE_STEPS = [0.7, 0.85, 1]` , `seasonMedalTier.ts:70`). The poster renders `'100% הצלחה בפנדלים'` (`he.ts:2461`).

Two methods, one Hebrew name. The club screen crowns `כתר הפנדלים / מלך הפנדלים` from a completely different algorithm. `src/utils/penaltyStats.ts:132–162` ranks by a **Wilson score lower bound** (`wilsonLowerBound` , lines 120–128, $z = 1.96$):

```
// penaltyStats.ts, pickLeader: rank by Wilson desc, then count, then attempts, then uid asc
score: wilsonLowerBound(count, attempts)
```

Its doc comment states the design intent explicitly: *"8/10 (0.49) outranks 1/1 (0.21), so a one-shot 100% no longer beats a proven high-volume scorer."* The season title does the exact opposite — a raw rate behind a hard floor, where 2/2 beats 2/3 and beats 8/10. On the same seven players, the club card and the season medal can name different people for the same crown, and since `he.ts:1370` points `communityStatsPenaltyKing` at the same `TITLE.penaltyKing` string, the two answers are labelled identically. There is also a second inconsistency in the same pair: `penaltyStats.pickLeader` breaks ties deterministically (`cand.userId < best.userId` , line 155) and returns exactly one leader, while `leaders()` shares. One crown, two rankers, two tie policies.

The same split exists for clean sheets. **Defect** — `cleansheetking-ranks-on-count-while-the-club-table-ranks-on-rate`. The season title is `leaders(eligible, p => p.cleanSheets)` — a raw count. The club's efficiency tab shows a sortable `cleanSheetPct` column (`CommunityStatsTable.tsx:251–252`) computed as `cleanSheets / csRounds` (`efficiencyStats.ts:80–83`), and the club's leaders card uses yet a third function, `leaderBy` (`CommunityStatsScreen.tsx:118–132, 463`), which is a strict-`>` scan that silently keeps the **first** row on the maximum — no shared ties, no floor beyond zero. On s1 the three disagree:

player	cleanSheets	csRounds	rate
alsobLSA (הלן צברי)	13	17	76.5%
1IdtNEjb (מתן לוי)	12	15	80.0%
YIZIKWBv (Eliran Tzabari)	11	13	84.6%

The season crowned `הלן צברי` on the count; the club's own rate column puts Eliran Tzabari top. Both are labelled `TITLE.cleanSheetKing = 'כתר השער הנקי'`.

The MVP

`mvp` is the mean of a player's *evening* scores: `mvpAvg = eveningScoreSum / eveningScoreCount` (`seasonRollover.ts:516–519`), with `0` substituted when the count is zero. The per-evening score comes from `eveningScoreServer` (`functions/src/eveningScoreCore.ts:34–65`), whose last line is:

```

if (gamesPlayed <= 0) return 6.0; // line 43 – the sentinel
...
const score = 6 + (weighted / 10) * 4;
return Math.round(Math.min(10, Math.max(6, score)) * 10) / 10; // line 64 – clamp to [6,10]

```

Three facts follow, and together they are the worst defect in this section:

1. The scale's **floor is 6.0, not 0** — a genuinely bad evening cannot score below 6.
2. **6.0 is also the sentinel** returned when `gamesPlayed <= 0`. `gamesPlayed` here is the player's *mini-game* count for the evening (`index.ts:6288`, `e. rounds`), which is zero for every evening a club runs on the plain timer. So the value that means "we recorded nothing about this player tonight" is numerically identical to the bottom of the scale.
3. **The sample size is never checked.** The eligibility gate tests `games` (evenings attended); `eveningScoreCount` is a different counter that only started incrementing when the accumulator shipped. A player with 19 attended evenings and one scored evening is compared on a mean of one.

Defect — `mvp-title-held-by-the-entire-club`. With the original `floor = 0`, the sentinel cleared the floor and every eligible player tied at exactly 6.0. Production `s1` is exactly that:

```

eveningScoreSum / eveningScoreCount = 18/3, 18/3, 18/3, 18/3, 18/3, 12/2, 12/2 → 6.0 × 7
awards.mvp = { value: 6, winners: [ ...all seven uids... ] }

```

Seven of the club's seven members hold כתר העונה at the value that means "nothing was recorded". A collection-group query over `seasonTitles` across the entire database returns **14 documents, of which 7 are mvp at value 6** — half of every season title ever awarded in this product is the no-data sentinel, shared by an entire club. Note also that `eveningScoreCount` is 2 or 3 against `games` of 17–19: even setting the sentinel aside, the "season average" was a mean over the last two or three evenings of a 22-evening season.

At HEAD this specific outcome is closed, by one argument added at line 221:

```

const MVP_SCALE_FLOOR = 6; // line 141
mvp: leaders(eligible, (p) => p.mvpAvg, (p) => p.uid, MVP_SCALE_FLOOR), // line 221

```

`best <= 6` now returns `null`, so a club whose evenings are all sentinels awards no MVP at all. Two residual problems remain, and neither is addressed:

- The sealed archive is not recomputed. `seasonSummary...__s1.awards.mvp`, the card's `winners`, and the seven `users/*/seasonTitles/*__mvp` documents all still exist in production and still render.
- A player with a *single* real mini-game evening (score, say, 6.3) and eighteen sentinel evenings averages $(6.3 + 18 \times 6.0) / 19 = 6.016$ — above the floor — and takes the title over teammates who are all flat 6.0. The floor removes the degenerate all-tie; it does not make the metric a season average of anything.

The medal tier

`src/utils/seasonMedalTier.ts` grades a *won* title's value into one of four metals. It is a pure presentation layer: nothing in the award computation reads it, and the tier is not stored — `SeasonHistoryScreen.tsx:172` recomputes it on every render from the card's `value` and `completedRounds` .

`SCALE` (lines 57–67) is a **total** `Record<SeasonTitleKey, Scale>` , deliberately, so a tenth title is a compile error rather than a silent bronze. Four kinds:

```
topScorer:      { kind: 'count', steps: [8, 16, 28] }
topAssister:    { kind: 'count', steps: [6, 12, 22] }
topWinner:      { kind: 'count', steps: [10, 20, 34] }
cleanSheetKing: { kind: 'count', steps: [5, 11, 20] }
deadlyDuo:      { kind: 'count', steps: [5, 11, 20] }
penaltyKing / penaltyKeeper: { kind: 'rate' }          // RATE_STEPS  = [0.7, 0.85, 1]
mostLoyal:      { kind: 'attendance' }                  // ATTENDANCE_STEPS = [0.6, 0.8, 1], value /
mvp:            { kind: 'eveningScore' }                // MVP_STEPS  = [6.6, 7.6, 8.8]
```

`step()` (lines 92–97) is `>=` at every stop, so `1.0` on a rate is platinum and `value === completedRounds` on attendance is platinum. `medalTier` guards `completedRounds <= 0 → 'bronze'` for the attendance scale only (line 120).

The count steps are absolute constants tuned against nothing documented, and they are graded against **mini-game** totals. On a club that plays ~37 mini-games a season, `topWinner` 's platinum at 34 wins and `topScorer` 's at 28 goals are not reachable; the season's entire club goal total was 27.

`MVP_STEPS` carries an honest comment (lines 72–86) admitting the original `[6.5, 7.5, 8.5]` was written against an assumed 1–10 scale that the clamp makes impossible, and that a "correction" to `[7, 8, 9]` made the title *harder* to lift off the floor. The current `[6.6, 7.6, 8.8]` are quarters of the real `[6, 10]` range; the club's own evening-standings leader sits at 6.93, which is silver. The ledger's `mvp-medal-can-never-leave-bronze` finding referred to the earlier constants.

`titleStreak` (lines 141–177) counts consecutive seasons with the same holder set, walking towards older seasons and stopping at a gap in `season.no` . It matches **by display name**, joined on `U+0000` , because a sealed `SeasonWinner` is `{key, names, value}` and carries no uid — a player who renames themselves silently ends their own streak, and the code says so in its comment rather than pretending otherwise. No production club has closed two seasons, so this path has never run against real data.

The one sealed season, end to end

Everything above, applied to `HhzIwmjMl1i5HS0GHt3p__s1` (מועדון שכחת שושי), season 1, 22 evenings, 37 mini-games, 7 members:

title	winner(s)	value	medal at HEAD	note
topScorer	הלן צברי	10 goals	silver (8/16/28)	37% of the club's 27 goals
topAssister	Nofar Tzabari	5 assists	bronze (6/12/22)	club total 13

title	winner(s)	value	medal at HEAD	note
mvp	all seven members	6.0	bronze	the sentinel; would be null at HEAD
topWinner	Lioz Madar	13 wins	silver (10/20/34)	mini-games, not evenings
mostLoyal	מתן לוי	19 evenings	gold (19/22 = 86%)	was platinum for one day; D = 19
cleanSheetKing	הלן צברי	13	gold (5/11/20)	rate ranking would name Eliran Tzabari
penaltyKing	הלן צברי	1.000	platinum	two kicks; flips to Nofar at D = 22
penaltyKeeper	הלן צברי	1.000	platinum	three faced, three saved
deadlyDuo	—	—	empty socket	best pair exchanged 2 directional assists, floor is > 2

Eight of nine awarded; one player holds four of the eight; one title is held by the entire club. seasonCards/..._s1.winners carries all eight with names frozen at close time, and fourteen seasonTitles documents were written across seven profiles.
